

LearnPrelude

every `prelude.py` name, by example

J. L. Clements III

September 16, 2026

One entry per prelude name, in `prelude.py`'s own order, drawn live from `prelude-doctests.py` – every example runs and passes, so this reference cannot drift from the code. The conceptually rich names carry DEEP DIVES: how the definition works, why it earns its place, the Haskell connection, and the sibling to contrast it with. Read a section, cover the outputs, predict; run `python3 prelude-doctests.py` to let the file grade itself.

Contents

1	combinators	2
2	pairs	6
3	arithmetic & logic	7
4	list basics	10
5	higher-order lists	14
6	folds	17
7	scans	21
8	streams	22
9	slicing & spans	25
10	sorting & searching	27
11	strings	30
12	containers	30
13	nodes	39
14	grids & windows	43
15	control	44

1 combinators

id — the identity function: returns its argument unchanged. Sounds useless; is everywhere. It is the "do nothing" you pass where a transformation is required – a sort key meaning "the value itself", the seed of `compose`.

```
>>> id(42)
42
>>> sortOn(id, [3, 1, 2])
[1, 2, 3]
```

Roll your own — the definition:

```
id = lambda x: x # shadows builtin id() by design
```

const — freezes a value into a function: `const(x)` returns a function that ignores its argument and always answers `x`. Use it to fill a callback slot with a constant.

```
>>> five = const(5)
>>> five("anything"), five(99)
(5, 5)
>>> map_(const('*'), [1, 2, 3])
['*', '*', '*']
```

Roll your own — the definition:

```
const = lambda x: lambda _: x # always returns x, ignoring the second argument
```

flip — swaps the two arguments of a function. An adapter: the function you have takes `(a, b)`, the slot you are filling supplies `(b, a)`. This is how the prelude derives `foldr` from `foldl`.

```
>>> sub(10, 1), flip(sub)(10, 1)
(9, -9)
>>> flip(pow)(3, 2) # pow(2, 3)
8
```

How it works: `flip(f)` returns a new function that hands its two arguments to `f` in the opposite order. Nothing is computed at flip time – it is pure plumbing, applied when the data arrives in the "wrong" order for the function you already have.

```
>>> pair = lambda a, b: (a, b)
>>> flip(pair)(1, 2)
(2, 1)
```

Why it earns its place: the prelude's own `foldr` is the showcase. `foldl`'s combiner takes (accumulator, element); a right fold's combiner takes (element, accumulator). Rather than build a second fold machine, `foldr = foldl` of the FLIPPED combiner over the reversed list – one adapter turned an existing engine into its mirror image.

```
>>> f = lambda x, acc: '(' + x + acc + ')'
>>> foldr(f, "abc", "*") == foldl(flip(f), list(reversed("abc")), "*")
True
```

Remember it as: when argument order is the only thing wrong, adapt – don't rewrite.

Roll your own — the definition:

```
flip = lambda f: (lambda x, y: f(y, x)) # flip f x y = f y x
```

curry — splits a two-argument function into two one-argument stages, so you can supply the first argument now and the second later.

```
>>> curry(add)(1)(2)
3
```

```
>>> add_ten = curry(add)(10)
>>> add_ten(5)
15
```

How it works: read the three lambdas inside-out, then watch a call peel one layer at a time. Each call binds ONE argument by closure – after `curry(add)(10)`, the 10 lives in the returned function's captured environment, and that half-applied function is a value you can name, store, and pass.

```
>>> curry(add)(10)(5)
15
>>> add_ten = curry(add)(10)
>>> add_ten(5), add_ten(90)
(15, 100)
```

Why it earns its place: the prelude's pipelines – `map_`, `filter_`, `find`, `compose` – all want ONE-argument functions in their function slot, while real logic usually takes two ("multiply by k", "is x greater than n?"). Currying is the adapter: close over the configuration argument now, hand the pipeline the unary stage it wants.

```
>>> gt = curry(lambda n, x: x > n)
>>> filter_(gt(10), [4, 11, 8, 40])
[11, 40]
>>> scale = curry(lambda k, x: k * x)
>>> map_(scale(3), [1, 2, 3])
[3, 6, 9]
```

The Haskell connection: in Haskell EVERY function is curried automatically – `add :: Int -> Int -> Int` really means `Int -> (Int -> Int)`, so `add 10` is ordinary code and `map (add 10) xs` falls out for free. Every arrow in a Haskell type is another layer of this doll.

Versus partial: both bind early. `partial` binds any number of arguments, all at once, when you have them; `curry` restructures the function so binding happens one call at a time, decided later by whoever holds it. In Python, `partial` is the cheaper everyday tool; `curry` is the pipeline-shaper and the reading key for Haskell signatures.

Remember it as: `curry` turns one two-argument function into two one-argument functions – and one-argument functions are the only thing pipelines eat.

Roll your own: three lambdas, each returning the next – every layer is a closure capturing one more argument until the innermost body finally has both. No state, no storage: the captured environment IS the data structure.

```
curry = lambda f: lambda x: lambda y: f(x, y)
```

uncurry — the reverse adaptation: a two-argument function becomes a one-argument function of a PAIR. Reach for it when your data is already (a, b) tuples and your function is not.

```
>>> uncurry(add)((1, 2))
3
>>> map_(uncurry(lambda a, b: a * b), [(2, 3), (4, 5)])
[6, 20]
```

Roll your own — the definition:

```
uncurry = lambda f: lambda p: f(*p) # inverse of curry: call f with a pair, not two args
```

partial — freezes any number of leading arguments, returning a function that takes the rest. Like `curry` but takes the arguments now, all at once.

```
>>> partial(pow, 2)(10) # pow(2, 10)
1024
>>> clamp_floor = partial(max, 0) # "never below zero"
>>> clamp_floor(-5), clamp_floor(7)
```

```
| (0, 7)
```

Roll your own — the definition:

```
| partial = lambda f, *bound: lambda *args: f(*bound, *args) # fix leading args of f, wait for
| the rest
```

on — builds a two-argument function that first sends BOTH arguments through a key function: `on(f, key)(x, y) == f(key(x), key(y))`. Its natural habitat is comparators: compare two records by one field.

```
| >>> on(add, len)("ab", "c") # len("ab") + len("c")
| 3
| >>> same_length = on(lambda a, b: a == b, len)
| >>> same_length("abc", "xyz"), same_length("ab", "abc")
| (True, False)
| >>> sortBy(on(sub, snd), [(1, 9), (2, 3)])
| [(2, 3), (1, 9)]
```

How it works: `on(f, key)` builds a two-argument function that routes BOTH inputs through `key` before combining them with `f`: `on(f, key)(x, y) == f(key(x), key(y))`. Combination logic and projection logic stay separate, so each is reusable on its own.

```
| >>> shorter = on(lambda a, b: a < b, len)
| >>> shorter("ab", "abc"), shorter("abc", "ab")
| (True, False)
```

Why it earns its place: comparator factories. Any "compare records by one field" collapses to `on(compare, field)` – Haskell's `sortBy (compare 'on' snd)` is exactly `sortBy(on(cmp, snd))` here, and reads as its own documentation.

```
| >>> sortBy(on(sub, snd), [('a', 9), ('b', 1), ('c', 5)])
| [('b', 1), ('c', 5), ('a', 9)]
```

Versus `sortOn`: when a per-element key exists, `sortOn(key, xs)` is simpler and faster (the key runs once per element). `on` shines when an API demands a genuine two-argument comparator, or when the combining function is something other than comparison.

Remember it as: compare the keys, not the things.

Roll your own — the definition:

```
| on = lambda f, g: lambda x, y: f(g(x), g(y)) # Data.Function: combine via a key -- cmp `on` fst
```

NOTHING — a private sentinel object meaning "no argument was supplied". Why not `None`? Because `None` is a legitimate VALUE a caller might pass; **NOTHING** can only mean "omitted", since nobody else has a reference to it. Always test with `'is'`. This is how `foldl` and `accumulate` tell an absent initial value from a real one.

```
| >>> x = NOTHING
| >>> x is NOTHING
| True
| >>> def greet(name=NOTHING):
| ...     return "hello, stranger" if name is NOTHING else "hello, " + name
| >>> greet()
| 'hello, stranger'
| >>> greet("ada")
| 'hello, ada'
```

How it works: `object()` creates a fresh, featureless object whose only property is its **IDENTITY** – nothing else in the program can ever be it. Testing `x is NOTHING` therefore asks exactly one question: "was I handed this precise sentinel?", which makes it a safe flag for "no argument was supplied".

Why not `None`, demonstrated: `None` is data (from `Maybe`'s whole job is handling it), so using it to also mean

"argument omitted" would blur two different situations. The sentinel keeps them apart: None is Nothing in the DATA; NOTHING is Nothing in the ARGUMENT LIST.

```
>>> def describe(x=NOTHING):
...     if x is NOTHING:
...         return "no argument"
...     return "got " + repr(x)
>>> describe()
'no argument'
>>> describe(None)
'got None'
```

Where the prelude leans on it: foldl and accumulate take an optional seed. With NOTHING as the default they can tell "seed omitted – use the first element" from "the seed IS None", which a None default could not.

Remember it as: None is a value; NOTHING is the absence of one.

Roll your own — the definition:

```
NOTHING = object() # Maybe's Nothing: "no arg given"; test with `is`
# hoisted ahead of foldl, which needs it as a sentinel
```

fromMaybe — lands a default: fromMaybe(d, x) is x unless x is None, in which case it is d. The standard way to finish a pipeline built on the None-is-Nothing convention (find, lookup, getpath, stripPrefix all return None on failure).

```
>>> fromMaybe(0, None), fromMaybe(0, 5)
(0, 5)
>>> fromMaybe('_', find(lambda c: c == 'q', "abc"))
'_'
>>> fromMaybe(99, lookup('b', [(('a', 1), ('b', 2))]))
2
```

Start from a problem you already have: find looks for something that might not be there, and when it is not, you are left holding None – which you cannot print to a user, add, or put in a report. So you write a backup plan:

```
>>> result = find(even, [3, 5, 7])
>>> if result is None:
...     result = 0
>>> result
0
```

fromMaybe is those three lines as one word. Argument order: the backup plan first, then the risky thing – said aloud it parses: "use 0 if this comes back empty".

```
>>> fromMaybe(0, find(even, [3, 5, 7]))
0
>>> fromMaybe(0, find(even, [3, 5, 8]))
8
```

Versus x or d: Python's or falls back on ANYTHING falsy – 0, "", [], False – not just None, so it silently destroys legitimate zero-ish answers. fromMaybe asks the only correct question: is this the absence marker?

```
>>> fromMaybe(99, 0), (0 or 99)
(0, 99)
```

Remember it as: ifNoneUse – the three-line backup plan as one word; or-with-a-conscience.

Roll your own — the definition:

```
fromMaybe = lambda d, x: d if x is None else x # Data.Maybe: default for every None-returning
tool
```

isJust — the None test as a predicate you can hand to other tools. The lambda it replaces, 'lambda v: v is not None', kept reappearing wherever a Maybe-producing step feeds a predicate slot; naming it also locks in the discipline: identity against None, never truthiness, so 0 and "" count as present.

```
>>> isJust(0), isJust(''), isJust(None)
(True, True, False)
>>> find(isJust, [None, None, 7, None])
7
>>> all(map(isJust, [1, 0, 'x']))
True
```

Roll your own — the definition:

```
isJust = lambda x: x is not None # Data.Maybe: None test as a handable predicate
```

isNothing — the mirror: True only for None. Counting failures reads as one word.

```
>>> isNothing(None), isNothing(0)
(True, False)
>>> len(filter_(isNothing, [None, 3, None]))
2
```

Together they finish the Maybe family: fromMaybe LANDS an optional value, mapMaybe/catMaybes FILTER optional values, isJust/isNothing TEST one. Haskell's Data.Maybe has all five under the same names.

```
>>> readings = [(1, None), (2, 7.5), (3, None)]
>>> fromMaybe(-1, find(isJust, map(snd, readings)))
7.5
```

Roll your own — the definition:

```
isNothing = lambda x: x is None # Data.Maybe: true when x is Nothing (None)
```

2 pairs

fst — the first element of a pair. Reads better than p[0] in code that is all tuples, and slots into higher-order positions.

```
>>> fst((1, 2))
1
>>> map_(fst, [(1, 'a'), (2, 'b')])
[1, 2]
```

Roll your own — the definition:

```
fst = lambda p: p[0] # first element of a pair
```

snd — the second element of a pair. The classic sort key for (key, count) items.

```
>>> snd((1, 2))
2
>>> sortOn(snd, [('a', 3), ('b', 1)])
[('b', 1), ('a', 3)]
```

Roll your own — the definition:

```
snd = lambda p: p[1] # second element of a pair
```

swap — exchanges a pair's elements. One honest use: inverting a dict by swapping its items.

```
>>> swap((1, 2))
```

```
| (2, 1)
| >>> dict(map_(swap, [('a', 1), ('b', 2)]))
| {1: 'a', 2: 'b'}
```

Roll your own — the definition:

```
| swap = compose(tuple, reversed) # Data.Tuple
```

3 arithmetic & logic

succ — successor: one more; on a character (Haskell’s Char instance of Enum), the next codepoint.

```
| >>> succ(41)
| 42
| >>> succ('a')
| 'b'
| >>> map_(succ, [1, 2, 3])
| [2, 3, 4]
```

Roll your own — the definition:

```
| succ = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1 # next: increments, advances
| chars
```

pred — predecessor: one less; characters step back one codepoint.

```
| >>> pred(43)
| 42
| >>> pred('b')
| 'a'
```

Roll your own — the definition:

```
| pred = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1 # Enum: numbers and Chars
```

even — is n divisible by two? A predicate shaped for filter__ and friends.

```
| >>> even(4), even(7)
| (True, False)
| >>> filter_(even, [1, 2, 3, 4])
| [2, 4]
```

Roll your own — the definition:

```
| even = lambda n: n % 2 == 0 # true for even numbers
```

odd — the complement of even.

```
| >>> odd(4), odd(7)
| (False, True)
| >>> partition(odd, [1, 2, 3, 4])
| ([1, 3], [2, 4])
```

Roll your own — the definition:

```
| odd = lambda n: n % 2 == 1 # true for odd numbers
```

not_ — logical negation as a function (“not” is a keyword, so it cannot be passed around; not_ can). Composes predicates.

```
| >>> not_(True)
| False
```

```
>>> is_odd = compose(not_, even)
>>> is_odd(3)
True
```

Roll your own — the definition:

```
not_ = lambda b: not b # `not` is a Python keyword
```

otherwise — just True, wearing Haskell’s name for the catch-all guard. Its habitat is the RULES TABLE: a long if/elif ladder rewritten as data – (predicate, result) pairs tried in order, find picking the first match. One care: Haskell guards are EVALUATED, so bare otherwise works there; a table’s guards are APPLIED to the argument, so wrap it as const(otherwise) – the guard that ignores its input and says yes. Result slots follow the same rule: fixed answers wear const, computed ones (like str below) go bare.

```
>>> otherwise
True
>>> RULES = [(lambda n: n % 15 == 0, const("FizzBuzz")),
...         (lambda n: n % 3 == 0, const("Fizz")),
...         (lambda n: n % 5 == 0, const("Buzz")),
...         (const(otherwise), str)]
>>> speak = lambda n: snd(find(lambda rule: fst(rule)(n), RULES))(n)
>>> map_(speak, [1, 3, 5, 15])
['1', 'Fizz', 'Buzz', 'FizzBuzz']
```

Roll your own — the definition:

```
otherwise = True # guard fallback: always true
```

signum — the sign of a number as -1, 0 or 1, computed by arithmetic on two comparisons – no branches. Also a ready-made comparator result.

```
>>> signum(-7), signum(0), signum(3)
(-1, 0, 1)
>>> sortBy(lambda a, b: signum(a - b), [3, 1, 2])
[1, 2, 3]
```

Roll your own — the definition:

```
signum = lambda x: (x > 0) - (x < 0) # -1, 0, or 1 by the sign of x
```

add — (+) as a value, for slots that want a function, not an operator: scanl1, zipWith, foldl.

```
>>> add(2, 3)
5
>>> scanl1(add, [1, 2, 3])
[1, 3, 6]
```

Roll your own — the definition:

```
add = lambda a, b: a + b # (+) as a value: scanl1(add, xs)
```

sub — (-) as a value, same habitat as add.

```
>>> sub(5, 2)
3
>>> zipWith(sub, [10, 20], [1, 2])
[9, 18]
```

Roll your own — the definition:

```
sub = lambda a, b: a - b # (-) as a value: zipWith(sub, a, b)
```

div — integer division that FLOORS (rounds toward negative infinity), like Haskell’s div and Python’s //.

The div/mod pair and the quot/rem pair agree on positives and split on negatives – interviews live in that gap.

```
>>> div(7, 2), div(-7, 2)
(3, -4)
```

Roll your own — the definition:

```
div = lambda a, b: a // b # floors, like Haskell div
```

mod — the remainder that matches div: its sign follows the DIVISOR, and $\text{div}(a,b)*b + \text{mod}(a,b) == a$ always holds.

```
>>> mod(7, 2), mod(-7, 2), mod(7, -2)
(1, 1, -1)
```

Roll your own — the definition:

```
mod = lambda a, b: a % b # remainder, sign follows b
```

quot — integer division that TRUNCATES (rounds toward zero), like Haskell’s quot and C’s /. Same as div for positives; differs by one for negatives.

```
>>> quot(7, 2), quot(-7, 2)
(3, -3)
```

Roll your own — the definition:

```
quot = lambda a, b: -(a // b) if (a < 0) != (b < 0) else a // b # truncates, like Haskell quot
```

rem — the remainder that matches quot: its sign follows the DIVIDEND.

```
>>> rem(7, 2), rem(-7, 2), rem(7, -2)
(1, -1, 1)
```

Roll your own — the definition:

```
rem = lambda a, b: a - b * quot(a, b) # remainder, sign follows a
```

quotRem — truncating quotient and remainder at once.

```
>>> quotRem(-7, 2)
(-3, -1)
```

Roll your own — the definition:

```
quotRem = lambda a, b: (quot(a, b), rem(a, b)) # (quot, rem) in one call
```

gcd — greatest common divisor by Euclid’s recursion, always ≥ 0 like Haskell’s. Recursion is safe here: the depth is at most ~90 even for 64-bit inputs (worst case: consecutive Fibonacci).

```
>>> gcd(48, 18)
6
>>> gcd(-4, 6), gcd(17, 5)
(2, 1)
```

Roll your own — the definition:

```
gcd = lambda a, b: abs(a) if b == 0 else gcd(b, a % b) #  $\geq 0$  like Haskell; depth  $\leq 90$ 
```

lcm — least common multiple via gcd, also always ≥ 0 ; defined as 0 when either input is 0.

```
>>> lcm(4, 6)
12
>>> lcm(-4, 6), lcm(0, 5)
(12, 0)
```

Roll your own — the definition:

```
| lcm = lambda a, b: abs(a // gcd(a, b) * b) if a and b else 0 # >= 0 like Haskell
```

halve — integer division by two, floored; the natural step for iterate and Newton-style halving.

```
|>>> halve(7), halve(-7)
(3, -4)
|>>> list(take(5, iterate(halve, 1024)))
[1024, 512, 256, 128, 64]
```

Roll your own — the definition:

```
| halve = lambda n: n // 2 # e.g. iterate(halve, 1024) -> 1024, 512, 256, ...
```

hypot — the Euclidean distance $\sqrt{x^2 + y^2}$. Note it returns a float.

```
|>>> hypot(3, 4)
5.0
```

Roll your own — the definition:

```
| hypot = lambda x, y: (x*x + y*y) ** 0.5 # straight-line distance
```

isqrt — floor square root using Newton's method on INTEGERS – no floats, so no precision loss on huge numbers; a negative input raises rather than lying.

```
|>>> isqrt(16), isqrt(17)
(4, 4)
|>>> isqrt(10**18)
1000000000
```

Roll your own: Newton's iteration on integers – repeatedly replace x with the average of x and $n // x$. It converges from above, so loop while the new estimate still shrinks; all arithmetic is $//$, so no float (and no float rounding lie) ever appears.

```
|def isqrt(n): # floor sqrt without floats (Newton)
|    if n < 0:
|        raise ValueError("isqrt of negative")
|    x = n
|    y = halve(x + 1)
|    while y < x:
|        x, y = y, halve(y + n // y)
|    return x if n else 0
```

4 list basics

head — the first element. A PARTIAL function: on an empty list it raises, so guard with `null()` or reach for `find/fromMaybe` when emptiness is possible.

```
|>>> head([1, 2, 3])
1
|>>> head([])
Traceback (most recent call last):
...
IndexError: list index out of range
```

Roll your own — the definition:

```
| head = lambda xs: xs[0] # first element
```

tail — everything after the first element. head and tail together are the grammar of structural recursion: do something with head, recurse on tail.

```
>>> tail([1, 2, 3])
[2, 3]
>>> tail([1])
[]
```

Roll your own — the definition:

```
tail = lambda xs: xs[1:] # all but the first element
```

init — everything before the last element (the mirror of tail).

```
>>> init([1, 2, 3])
[1, 2]
```

Roll your own — the definition:

```
init = lambda xs: xs[:-1] # all but the last element
```

last — the final element. Partial, like head.

```
>>> last([1, 2, 3])
3
```

Roll your own — the definition:

```
last = lambda xs: xs[-1] # last element
```

null — is the list empty? The guard that makes head/tail safe to use.

```
>>> null([]), null([1])
(True, False)
```

Roll your own — the definition:

```
null = lambda xs: len(xs) == 0 # true for an empty list
```

elem — membership test, argument order (needle, haystack).

```
>>> elem(2, [1, 2, 3])
True
>>> elem('q', "abc")
False
```

Roll your own — the definition:

```
elem = lambda x, xs: x in xs # true if x is in xs
```

notElem — non-membership.

```
>>> notElem(9, [1, 2, 3])
True
```

Roll your own — the definition:

```
notElem = lambda x, xs: x not in xs # true if x is not in xs
```

replicate — n copies of one value, as a list.

```
>>> replicate(3, 'x')
['x', 'x', 'x']
>>> replicate(0, 'x')
[]
```

Roll your own — the definition:

```
| replicate = lambda n, x: [x] * n # x repeated n times
```

drop — discard the first *n* elements, keep the rest; finite lists only (it materialises the input), and a negative *n* drops nothing.

```
| >>> drop(2, [1, 2, 3, 4])
| [3, 4]
| >>> drop(10, [1, 2]), drop(-3, [1, 2])
| ([], [1, 2])
```

Roll your own — the definition:

```
| drop = lambda n, xs: list(xs)[max(n, 0):] # finite lists; n<0 drops none
```

splitAt — cut a list into (first *n*, rest) in one call; a negative *n* splits at the front. The pieces glue back with `+`, which is why rotations and chunking fall out of it.

```
| >>> splitAt(2, [1, 2, 3, 4])
| ([1, 2], [3, 4])
| >>> splitAt(0, [1, 2]), splitAt(-2, [1, 2])
| (([], [1, 2]), ([], [1, 2]))
```

Roll your own — the definition:

```
| splitAt = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs) # n<0 splits at the front
```

nub — deduplicate, FIRST occurrence wins, order preserved. The trick: `dict.fromkeys` builds a dict (whose keys are unique and insertion-ordered) and throws away the values. Elements must be hashable. Compare `set(xs)`, which destroys order.

```
| >>> nub([3, 1, 3, 2, 1])
| [3, 1, 2]
| >>> nub("mississippi")
| ['m', 'i', 's', 'p']
```

Roll your own — the definition:

```
| nub = compose(list, dict.fromkeys) # unique; first seen wins
```

lookup — the first value paired with a key in an association list (a list of (key, value) tuples), or `None` if absent. First match wins, so an assoc list can shadow like a scope chain.

```
| >>> lookup('b', [('a', 1), ('b', 2)])
| 2
| >>> lookup('a', [('a', 1), ('a', 99)])
| 1
| >>> lookup('z', [('a', 1)]) is None
| True
```

Roll your own — the definition:

```
| lookup = lambda k, pairs: next((v for kk, v in pairs if kk == k), None) # Nothing -> None
```

zip_ — pair two iterables positionally, stopping at the shorter; returns a real list (the builtin `zip` returns a lazy iterator), and either side may be any iterable, streams included.

```
| >>> zip_([1, 2], [10, 20, 30])
| [(1, 10), (2, 20)]
| >>> zip_("ab", count(0))
| [('a', 0), ('b', 1)]
```

Roll your own — the definition:

```
| zip_ = lambda a, b: list(zip(a, b)) # shortest wins, any iterable
```

zip3 — the three-list version.

```
| >>> zip3([1, 2], ['a', 'b'], [True, False])
| [(1, 'a', True), (2, 'b', False)]
```

Roll your own — the definition:

```
| zip3 = lambda a, b, c: list(zip(a, b, c)) # zip three lists together
```

unzip — a list of pairs back into a pair of lists; the inverse of `zip_`.

```
| >>> unzip([(1, 'a'), (2, 'b')])
| ([1, 2], ['a', 'b'])
| >>> unzip([])
| ([], [])
```

Roll your own — the definition:

```
| unzip = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], []) # inverse of zip: pairs ->
| two lists
```

transpose — rows become columns, RAGGED-SAFE exactly like Haskell's: short rows simply drop out of later columns; nothing is silently truncated.

```
| >>> transpose([[1, 2, 3], [4, 5, 6]])
| [[1, 4], [2, 5], [3, 6]]
| >>> transpose([[1, 2, 3], [4, 5]])
| [[1, 4], [2, 5], [3]]
| >>> transpose([])
| []
```

Rows may be any iterables, even one-shot ones – the rows are materialised first, so a generator of iterators transposes like a list of lists (without that step the generator is spent by the length scan):

```
| >>> transpose(iter([1, 2]) for _ in range(2))
| [[1, 1], [2, 2]]
| >>> transpose(["abc", "de"])
| [['a', 'd'], ['b', 'e'], ['c']]
```

Roll your own: materialise the rows, find the longest, then for each column index keep `xs[i]` only from rows that reach it – the `'if i < len(xs)'` filter is precisely where short rows drop out of later columns instead of truncating everyone.

```
| def transpose(xss): # Data.List: rows <-> cols, RAGGED-SAFE like Haskell
|     xss = [list(xs) for xs in xss] # any iterables in (rows may be one-shot); materialise once
|     n = max(map(len, xss), default=0) # short rows just drop out of later columns (no
|     truncation)
|     return [[xs[i] for xs in xss if i < len(xs)] for i in range(n)]
```

enum — pair every element with its index: a list-returning `enumerate`. The shape that feeds index-aware folds (two_sum's seen-dict, next_greater's stack).

```
| >>> enum(['a', 'b'])
| [(0, 'a'), (1, 'b')]
| >>> enum(['a', 'b'], start=1)
| [(1, 'a'), (2, 'b')]
```

Roll your own — the definition:

```
| enum = lambda xs, start=0: zip_(list(range(start, start + len(xs))), xs) # pairs each x with
| its index
```

pairwise — every element with its RIGHT neighbour: `zip xs (tail xs)`. The tool for any rule about adjacency – deltas, local comparisons, Roman numeral subtraction.

```
>>> pairwise([1, 4, 9])
[(1, 4), (4, 9)]
>>> map_(lambda p: p[1] - p[0], pairwise([3, 10, 6]))
[7, -4]
```

Roll your own — the definition:

```
| pairwise = lambda xs: zip_(xs, xs[1:]) # zip xs (tail xs)
```

isPrefixOf — does `xs` start with `p`? Works on strings and lists alike (both are normalised through `list()`).

```
>>> isPrefixOf("re", "rebuild")
True
>>> isPrefixOf([1, 2], [1, 2, 3])
True
>>> isPrefixOf("", "anything")
True
```

Roll your own — the definition:

```
| isPrefixOf = lambda p, xs: list(xs[:len(p)]) == list(p) # Data.List; strings or lists
```

isSuffixOf — does `xs` end with `p`? Implemented by slicing from `len(xs) - len(p)`, NOT by `xs[-len(p):]`, because when `p` is empty that slice is `xs[-0:]` == the WHOLE list. A scar worth keeping visible.

```
>>> isSuffixOf("ing", "folding")
True
>>> isSuffixOf("", "x")
True
>>> isSuffixOf("abc", "bc")
False
```

Roll your own — the definition:

```
| isSuffixOf = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p) # not [-len(p):]: [-0:] is ALL
```

stripPrefix — Just the rest after the prefix, or Nothing: returns `xs[len(p):]` when the prefix matches, `None` when it does not. Pairs with `fromMaybe` and `mapMaybe`.

```
>>> stripPrefix("re", "rebuild")
'build'
>>> stripPrefix("un", "rebuild") is None
True
>>> mapMaybe(partial(stripPrefix, "err:"), ["err:a", "ok:b", "err:c"])
['a', 'c']
```

Roll your own — the definition:

```
| stripPrefix = lambda p, xs: xs[len(p):] if isPrefixOf(p, xs) else None # Just the rest, or
    Nothing
```

5 higher-order lists

map_ — apply a function to every element; returns a real list, eagerly (the builtin `map` is lazy). The workhorse transformation.

```
>>> map_(lambda x: x * x, [1, 2, 3])
[1, 4, 9]
```

```
|>>> map_(str.upper, ["ab", "cd"])
|['AB', 'CD']
```

Roll your own — the definition:

```
|map_ = lambda f, xs: [f(x) for x in xs] # apply f to every element
```

filter_ — keep the elements satisfying a predicate. Scans the WHOLE list; if you only want the first hit, that is `find`.

```
|>>> filter_(even, [1, 2, 3, 4])
|[2, 4]
|>>> filter_(str.isdigit, "a1b22")
|['1', '2', '2']
```

Roll your own — the definition:

```
|filter_ = lambda cond, xs: [x for x in xs if cond(x)] # keep elements where cond holds
```

find — the first element satisfying the predicate, else `None`. Lazy: it stops at the first hit, which folds cannot do, and therefore works even on infinite streams. Finish with `fromMaybe` if you need a default.

```
|>>> find(even, [1, 3, 4, 6])
|4
|>>> find(even, [1, 3]) is None
|True
|>>> find(lambda x: x % 7 == 0, count(1))
|7
```

How it works: `find` wraps a generator expression in `next()` with a `None` default. The generator is the point — elements are tested one at a time, on demand, and the search STOPS at the first hit. Nothing after the match is ever examined.

Why it earns its place: folds and `filter_` consume the whole list; they cannot stop early. `find` can — which makes it the right tool the moment the word "first" appears in a statement, and the only tool that survives an infinite stream. Land defaults with `fromMaybe`.

```
|>>> find(lambda x: x * x > 50, count(1))
|8
|>>> fromMaybe('?', find(str.isdigit, "abc7de"))
|'7'
```

Versus `filter_`: `filter_` answers "which ones?"; `find` answers "who was first, if anyone?". Taking `filter_(p, xs)[0]` instead does wasted work and explodes when nothing matches; `find` does neither.

Remember it as: `find` is the early exit that folds cannot perform.

Roll your own: a generator expression handed to `next()` with a default. The generator is what makes it lazy — elements are produced one at a time, so the first hit stops everything — and the default is what makes it total.

```
|find = lambda cond, xs: next((x for x in xs if cond(x)), None) # first match, else None
```

partition — split into (satisfy, don't) in ONE pass, both sides in original order; the predicate runs exactly once per element, so it may be expensive or even stateful.

```
|>>> partition(even, [1, 2, 3, 4])
|([2, 4], [1, 3])
|>>> partition(str.isalpha, "a1b2")
|(['a', 'b'], ['1', '2'])
```

Roll your own: two buckets and one conditional append — (yes if `cond(x)` else no).`append(x)`. The conditional expression picks the LIST, then the append mutates it; one pass, exactly one predicate call per element.

```
|def partition(cond, xs): # (keepers, rest) in ONE pass; cond called once per element
```

```

    yes, no = [], []
    for x in xs:
        (yes if cond(x) else no).append(x)
    return (yes, no)

```

mapMaybe — map a function that returns a value or None, and keep only the values: parse-and-filter in ONE pass. The shape for messy input – write a parser that answers None for garbage, then mapMaybe it over everything. dict.get is already such a function.

```

>>> mapMaybe(lambda s: int(s) if s.isdigit() else None, ["3", "x", "7"])
[3, 7]
>>> mapMaybe({'a': 1, 'b': 2}.get, ['a', 'x', 'b'])
[1, 2]

```

How it works: run f over every element; f answers a VALUE when it can make sense of the input and None when it cannot; keep only the values. Map and filter fused into one pass, with the parser itself deciding what survives.

```

>>> parse_int = lambda s: int(s) if s.lstrip('-').isdigit() else None
>>> mapMaybe(parse_int, ["12", "x", "-3", ""])
[12, -3]

```

Why it earns its place: messy input. The alternative is a loop with an if, a flag and an append – three chances for a bug. Here the contract is crisp: write parse(item) -> value-or-None once, and mapMaybe handles every item. dict.get is already such a function, so lookups that may miss compose directly.

```

>>> legend = {'a': 1, 'b': 2}
>>> mapMaybe(legend.get, "cabbage")
[1, 2, 2, 1]

```

The Haskell connection: Data.Maybe's mapMaybe, with None standing in for Nothing – the "parse, don't validate" discipline: turn questionable data into clean data at the boundary, and everything downstream stops worrying.

Remember it as: map with a parser, drop the Nothings, one pass.

Roll your own — the definition:

```

mapMaybe = lambda f, xs: [y for y in (f(x) for x in xs) if y is not None]

```

catMaybes — keep the non-None values; mapMaybe with the identity.

```

>>> catMaybes([1, None, 2, None])
[1, 2]

```

Roll your own — the definition:

```

catMaybes = partial(filter_, isJust) # mapMaybe id

```

concat — flatten ONE level of nesting.

```

>>> concat([[1, 2], [3], []])
[1, 2, 3]
>>> concat(["ab", "cd"])
['a', 'b', 'c', 'd']

```

Roll your own — the definition:

```

concat = lambda xss: [x for xs in xss for x in xs] # flatten one level of nesting

```

concatMap — map each element to a LIST, and pool all the results: concat after map_, fused. The backbone of enumeration – "for every choice, produce all outcomes".

```

>>> concatMap(lambda w: [w, w.upper()], ["a", "b"])

```

```

['a', 'A', 'b', 'B']
>>> concatMap(lambda x: [x] * x, [1, 2, 3])
[1, 2, 2, 3, 3, 3]
>>> concatMap(lambda x: [x, -x], [1, 2]) == concat(map_(lambda x: [x, -x], [1, 2]))
True

```

Roll your own — the definition:

```

concatMap = lambda f, xs: [y for x in xs for y in f(x)] # map f over xs, then flatten

```

starmap — map a MULTI-argument function over a list of argument tuples: `map_` composed with `uncurry`.

```

>>> starmap(lambda a, b: a * b, [(2, 3), (4, 5)])
[6, 20]
>>> starmap(pow, [(2, 3), (3, 2)])
[8, 9]

```

Roll your own — the definition:

```

starmap = lambda f, pairs: [f(*p) for p in pairs] # map . uncurry

```

zipWith — combine two lists elementwise with a function; stops at the shorter. `zip_` is just `zipWith(make-a-pair)`.

```

>>> zipWith(add, [1, 2], [10, 20])
[11, 22]
>>> zipWith(lambda a, b: a > b, [3, 1], [2, 5, 9])
[True, False]

```

Roll your own — the definition:

```

zipWith = lambda f, a, b: [f(x, y) for x, y in zip(a, b)] # combine two lists elementwise

```

zipWith3 — the three-list version.

```

>>> zipWith3(lambda a, b, c: a + b + c, [1], [2], [3])
[6]

```

Roll your own — the definition:

```

zipWith3 = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)] # combine three lists
                               elementwise

```

cross — every pairing of two lists (the cartesian product). For the product of a list with ITSELF `n` times, see `replicateM`.

```

>>> cross([1, 2], "ab")
[(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
>>> len(cross(range(3), range(4)))
12

```

Roll your own — the definition:

```

cross = lambda a, b: [(x, y) for x in a for y in b] # cartesian; `product` is the fold

```

6 folds

foldl — THE left fold, and the prelude's most important definition. It is exactly this loop: `acc = base; for x in xs: acc = f(acc, x); return acc`. The combiner takes (ACCUMULATOR, element) – accumulator first. Argument order of `foldl` itself: (f, xs, base), with base optional – omitted, the first element seeds the accumulator (and an empty list is then an error).

```

>>> foldl(lambda acc, x: acc * 10 + x, [1, 2, 3], 0)
123
>>> foldl(max, [3, 9, 2])          # no base: 3 seeds it
9
>>> foldl(add, [], 0)              # empty + base: the base comes back
0
>>> foldl(lambda acc, w: acc + len(w), ["ab", "c"], 0)
3

```

How it works: foldl is precisely this loop, given a name and a contract:

```

>>> def foldl_spelled_out(f, xs, base):
...     acc = base
...     for x in xs:
...         acc = f(acc, x)
...     return acc
>>> foldl_spelled_out(add, [1, 2, 3], 0) == foldl(add, [1, 2, 3], 0)
True

```

Two disciplines hide in the signature. The combiner takes (ACCUMULATOR, element) – accumulator first, always; reversing them is the classic fold bug. And the optional base is guarded by NOTHING, so the fold can tell "no seed – use the first element" from "the seed is None".

Why it earns its place: the fold is the universal list consumer. sum, max, reverse, "build a dict", "run a state machine over events" – each is one choice of combiner and seed. When you can say "carry THIS state, update it THIS way per element", the loop is already written.

```

>>> foldl(lambda acc, x: [x] + acc, [1, 2, 3], [])          # reverse is a fold
[3, 2, 1]
>>> foldl(lambda acc, w: acc + len(w), ["ab", "cde"], 0)
5

```

Remember it as: a fold is a for-loop whose state has been given a contract.

Roll your own: seed the accumulator – base if one was given, else the first element (the NOTHING guard is what lets None be a legitimate seed) – then a single for-loop folding $acc = f(acc, x)$. The entire contract is which side the accumulator sits on.

```

def foldl(f, xs, base=NOTHING): # THE left fold; Python buried its own in functools as reduce
    it = iter(xs)
    if base is NOTHING:
        try:
            acc = next(it)
        except StopIteration:
            raise TypeError("fold of empty sequence with no initial value")
    else:
        acc = base
    for x in it:
        acc = f(acc, x)
    return acc

```

foldl1 — foldl seeded by the first element, spelled out. Use when the accumulator has the same type as the elements and emptiness cannot happen.

```

>>> foldl1(add, [1, 2, 3])
6
>>> foldl1(min, [3, 1, 2])
1

```

Roll your own — the definition:

```

foldl1 = foldl # foldl seeded by the first element

```

foldr — the RIGHT fold: nests from the right, *f* takes (element, ACCUMULATOR) – element first, mirror of *foldl*. Implemented as *foldl* of the flipped function over the reversed list: iterative and stack-safe, an identity that only holds because Python is strict. The parenthesisatation example makes the direction visible.

```
>>> foldr(lambda x, acc: '(' + x + '+' + acc + ')', "abc", "z")
'(a+(b+(c+z)))'
>>> foldl(lambda acc, x: '(' + acc + '+' + x + ')', "abc", "z")
'(((z+a)+b)+c)'
>>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])
[1, 2, 3]
```

How it works: the mirror image – the combiner takes (element, ACCUMULATOR) and the nesting grows from the RIGHT: $f(a, f(b, f(c, z)))$. The prelude does not write a second recursion: $\text{foldr} = \text{foldl}(\text{flip}(f), \text{reversed}(xs), \text{base})$. Flip fixes the argument order, reversing fixes the traversal, and the existing engine runs backwards.

The honesty clause: that identity only holds because Python is STRICT. Haskell's *foldr* can process infinite lists – laziness lets *f* decide whether the rest is ever needed; ours materialises and reverses the list first, so it cannot. Same name, same finite behaviour, different superpower.

Why it earns its place: building right-to-left. Consing onto a front is natural from the right – section 13's *from_list* IS a *foldr* of *ListNode*, and here it is, derived by hand:

```
>>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])          # rebuild, order kept
[1, 2, 3]
>>> to_list(foldr(ListNode, [1, 2, 3], None))              # from_list, spelled out
[1, 2, 3]
```

Remember it as: *foldl* eats left-to-right; *foldr* thinks right-to-left – and in strict Python it is *foldl* wearing flip and a reversal.

Roll your own — the definition:

```
foldr = lambda f, xs, base: foldl(flip(f), reversed(list(xs)), base) # THE right fold, via foldl
```

foldr1 — right fold seeded by the LAST element.

```
>>> foldr1(sub, [1, 2, 3])  # 1 - (2 - 3)
2
```

Roll your own — the definition:

```
foldr1 = lambda f, xs: foldl(flip(f), reversed(list(xs))) # foldr seeded by the last element
```

product — multiply everything: the numeric fold, seeded with 1 so an empty product is 1.

```
>>> product([2, 3, 4])
24
>>> product([])
1
```

Roll your own — the definition:

```
product = lambda xs: foldl(lambda a, x: a * x, xs, 1) # the numeric fold
```

compose — fold any number of functions into one; the RIGHTMOST runs first, like mathematical composition $f(g(x))$.

```
>>> compose(str.strip, str.upper)(" hi ")
'HI'
>>> f = compose(succ, succ, succ)
>>> f(0)
3
```

How it works: a fold over the functions themselves – neighbours merge into $\lambda x: f(g(x))$, seeded with the identity, so `compose()` with no arguments is `id`. The RIGHTMOST function runs first, matching mathematical $(f \cdot g)(x) = f(g(x))$.

```
>>> compose()(42) == id(42)
True
>>> compose(len, str.strip)(" hi ")      # strip first, then len
2
```

Why it earns its place: it turns a pipeline into a VALUE. A composed function can be named, mapped, stored in a rules table – processing steps become data you can shuffle. The constraint is the one curry answers: every stage must be unary, which is why `compose`-heavy code leans on `curry` and `partial` to shape its stages.

```
>>> normalize = compose(str.lower, str.strip)
>>> map_(normalize, [" Hi ", " YO "])
['hi', 'yo']
```

Remember it as: `foldr (.) id` – a pipeline you can hold in your hand.

Roll your own: it IS `foldr (.) id` – `foldr` over the functions with `x` as the seed, each stage applied to the accumulated result: `foldr(lambda f, acc: f(acc), fns, x)`. Because the prelude's `foldr` is an iterative loop underneath, stack depth stays constant no matter how many stages you compose.

```
compose = lambda *fns: lambda x: foldr(lambda f, acc: f(acc), fns, x) # foldr (.) id: RIGHTMOST
first
```

subsequences — every subset of the elements (the powerset), built by a fold in which each element DOUBLES the accumulator: keep every existing subset, and every existing subset extended by the newcomer. 2^n results – a brute-force license for `n` up to about 20, and you should say that bound out loud before using it.

```
>>> subsequences([1, 2])
[[], [1], [2], [1, 2]]
>>> subsequences("ab")
[[], ['a'], ['b'], ['a', 'b']]
>>> len(subsequences("abcd"))
16
```

How it works: watch the accumulator double. Start from `[]` – one subset, the empty one. Each element `x` rewrites `acc` as `acc + [s + [x] for s in acc]`: every existing subset survives (`x` left out), and every existing subset reappears extended by `x` (`x` taken). `n` elements, `n` doublings, 2^n subsets, in a stable order.

```
>>> acc = []
>>> acc = acc + [s + [1] for s in acc]; acc
[[], [1]]
>>> acc = acc + [s + [2] for s in acc]; acc
[[], [1], [2], [1, 2]]
```

Why it earns its place: it is a brute-force LICENSE. At `n <= 20`, checking every subset is a few million cheap operations – honest, exhaustive, and unbeatable to debug. Say the bound out loud (" 2^{15} is 32768 – fine") and enumerate; cleverness can come later if the bound grows.

```
>>> best = maxOn(sum, [s for s in subsequences([3, -1, 4, -2]) if sum(s) <= 6])
>>> sum(best)
6
```

Remember it as: the powerset fold – each element doubles the world.

Roll your own — the definition:

```
subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]])
```

replicateM — every length-`n` word over an alphabet: cross applied `n` times, $|xs|^n$ results. The other brute-force license – all bitstrings, all dice rolls, all assignments of `n` slots.

```
>>> replicateM(2, "ab")
[['a', 'a'], ['a', 'b'], ['b', 'a'], ['b', 'b']]
>>> len(replicateM(3, [0, 1]))
8
```

Roll your own — the definition:

```
replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n),
    [[]])
```

7 scans

accumulate — the generator behind the scans: yields the running combination of everything seen so far. With no function it sums; 'initial' seeds it. This is Python's `itertools` name with Haskell's scan semantics.

```
>>> list(accumulate([1, 2, 3]))
[1, 3, 6]
>>> list(accumulate([1, 2], initial=10))
[10, 11, 13]
>>> list(accumulate([3, 1, 2], min))
[3, 1, 1]
```

Roll your own: the same skeleton as `foldl`, but a **GENERATOR** – yield the accumulator once before the loop, then yield again after every combine. The early bare return on an empty, seedless input is what makes an empty scan come back empty instead of raising.

```
def accumulate(xs, f=None, initial=NOTHING): # scanl / scanl1
    if f is None:
        f = add
    it = iter(xs)
    if initial is NOTHING:
        try:
            acc = next(it)
        except StopIteration:
            return
    else:
        acc = initial
    yield acc
    for x in it:
        acc = f(acc, x)
        yield acc
```

scanl — a fold that **KEEPS ITS HISTORY**: returns every intermediate accumulator, starting with the seed, so the result has `len(xs)+1` elements and its last element equals the `foldl`. Argument order `(f, z, xs)` – seed in the middle, Haskell's order, unlike `foldl`'s trailing base. Prefix sums are the canonical scan.

```
>>> scanl(add, 0, [3, 1, 4])
[0, 3, 4, 8]
>>> last(scanl(add, 0, [1, 2, 3])) == foldl(add, [1, 2, 3], 0)
True
```

The DP secret: many celebrated one-pass algorithms are scans in a trench coat, because a scan IS a dynamic-programming table – entry `i` holds the answer for the prefix ending at `i`. Kadane's famous maximum-subarray "trick" is one `scanl1` and a `max`:

```
>>> xs = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
>>> max(scanl1(lambda best, x: max(x, best + x), xs))
6
```

Prefix sums, the other superpower: scan once with (+) and every range sum becomes two reads, `pref[j] - pref[i]` – $O(n)$ window questions collapse to $O(1)$.

```
>>> pref = scanl(add, 0, [3, 1, 4, 1, 5])
>>> pref[4] - pref[1]          # sum of elements 1..3
6
```

Remember it as: when you need the state at EVERY step, one scan replaces n folds.

Roll your own — the definition:

```
| scanl = lambda f, z, xs: list(accumulate(xs, f, initial=z)) # foldl, keeping every accumulator
```

scanl1 — the history-keeping fold seeded by the first element; `len(xs)` results. Running maxima and minima live here.

```
>>> scanl1(min, [3, 1, 2])
[3, 1, 1]
>>> scanl1(max, [1, 3, 2, 5])
[1, 3, 3, 5]
```

Roll your own — the definition:

```
| scanl1 = lambda f, xs: list(accumulate(xs, f)) # scanl with no explicit base
```

scanr — the suffix scan: folds from the RIGHT, showing every suffix's result; the first element is the full fold, the last is the seed. Suffix sums for product_except_self-style problems.

```
>>> scanr(add, 0, [1, 2, 3])
[6, 5, 3, 0]
```

Roll your own — the definition:

```
| scanr = lambda f, z, xs: list(reversed(scanl(flip(f), z, list(reversed(list(xs))))) # foldr,
    keeping every acc
```

scanr1 — suffix scan seeded by the last element.

```
>>> scanr1(add, [1, 2, 3])
[6, 5, 3]
```

Roll your own — the definition:

```
| scanr1 = lambda f, xs: list(reversed(scanl1(flip(f), list(reversed(list(xs))))) # scanr with
    no base
```

8 streams

count — the infinite arithmetic sequence `start, start+step, ...` Never consume it whole: pair it with `take`, `takewhile` or `find`, which stop.

```
>>> take(3, count(10))
[10, 11, 12]
>>> take(3, count(0, 5))
[0, 5, 10]
```

Roll your own — the definition:

```
| def count(start=0, step=1): # [start, start+step ..]
    n = start
    while True:
        yield n
        n += step
```

repeat — one value forever, or exactly n times with the second argument.

```
>>> take(2, repeat('x'))
['x', 'x']
>>> list(repeat('x', 3))
['x', 'x', 'x']
>>> list(repeat('x', 0))
[]
```

Roll your own — the definition:

```
def repeat(x, n=None): # yield x forever, or n times if given
    if n is None:
        while True:
            yield x
    else:
        for _ in range(n):
            yield x
```

cycle — a finite list looped forever; an EMPTY list raises immediately (Haskell errors too – the alternative is a silent infinite hang).

```
>>> take(5, cycle([1, 2]))
[1, 2, 1, 2, 1]
```

Roll your own — the definition:

```
def cycle(xs): # repeat xs forever
    xs = list(xs)
    if not xs:
        raise ValueError("cycle: empty list") # Haskell errors too; the alternative is a silent
        hang
    while True:
        yield from xs
```

iterate — x, f(x), f(f(x)), ... forever: repeated application as a stream. Any deterministic state machine becomes iterate on its step function – Fibonacci is iterate on a pair.

```
>>> take(4, iterate(lambda x: 2 * x, 1))
[1, 2, 4, 8]
>>> step = lambda p: (snd(p), fst(p) + snd(p))
>>> map_(fst, take(6, iterate(step, (0, 1))))
[0, 1, 1, 2, 3, 5]
```

Roll your own — the definition:

```
def iterate(f, x): # iterate f x = [x, f x, f (f x), ...]
    while True:
        yield x
        x = f(x)
```

unfoldr — iterate's FINITE twin, and the cure for threading (state, acc) tuples through loops. You supply f(seed) returning either (value, seed') to emit a value and continue, or None to stop; unfoldr collects the values. Anything that "peels" a structure – digits, chunks, parent chains – is an unfoldr.

```
>>> unfoldr(lambda m: None if m == 0 else (m % 10, m // 10), 407)
[7, 0, 4]
>>> unfoldr(lambda n: None if n == 0 else (n, n - 1), 5)
[5, 4, 3, 2, 1]
>>> unfoldr(const(None), "seed") # stop immediately: empty
[]
```

How it works: the step function carries a two-outcome contract – $f(\text{seed})$ returns $(\text{value}, \text{next_seed})$ to emit and continue, or `None` to stop – and `unfoldr` collects what was emitted. It is the fold REVERSED: a fold consumes a list into a summary; an unfold grows a list out of a seed. (Category theory says anamorphism; interviewers say "generate the sequence".)

Why it earns its place: it retires the $(\text{state}, \text{accumulator})$ tuple you would otherwise thread through a while loop. Anything that PEELS – digits off a number, chunks off a list, parents up a tree, a Collatz trajectory – states its entire logic in the step function.

```
>>> unfoldr(lambda n: None if n == 1 else (n, n // 2 if even(n) else 3 * n + 1), 6)
[6, 3, 10, 5, 16, 8, 4, 2]
```

Versus `iterate`: `iterate` is the infinite cousin – an unfold whose step never says `None`, consumed with `take`. Reach for `iterate` when the stream is conceptually endless, `unfoldr` when the seed itself knows where the end is.

Remember it as: fold tears down; unfold builds up; the seed carries the future.

More unfolds, same two-outcome contract – peeling chunks off a list, bits off a number, walking a parent chain to its root:

```
>>> unfoldr(lambda xs: None if xs == [] else (xs[:2], xs[2:]), [1, 2, 3, 4, 5])
[[1, 2], [3, 4], [5]]
>>> unfoldr(lambda m: None if m == 0 else (m % 2, m // 2), 13)          # bits, LSB first
[1, 0, 1, 1]
>>> parents = {'c': 'b', 'b': 'a', 'a': None}
>>> unfoldr(lambda n: None if n is None else (n, parents[n]), 'c')    # ancestry
['c', 'b', 'a']
```

And the seed can carry several facts at once – here $(\text{current}, \text{next}, \text{remaining})$ generates Fibonacci with its own countdown built in:

```
>>> unfoldr(lambda s: None if s[2] == 0 else (s[0], (s[1], s[0] + s[1], s[2] - 1)), (0, 1, 8))
[0, 1, 1, 2, 3, 5, 8, 13]
```

Roll your own: prime the pump with $\text{step} = f(\text{seed})$, then loop while step is not `None` – append the value, feed the new seed back to f . The two-outcome contract lives entirely in f ; `unfoldr` is just the loop that honours it, plus the list that collects what was emitted.

```
def unfoldr(f, seed): # Data.List unfoldr: f(seed) -> None | (value, seed')
    out = [] # iterate's finite twin: grows a list until f says stop --
    step = f(seed) # no more threading (state, acc) tuples through until
    while step is not None:
        val, seed = step
        out.append(val)
        step = f(seed)
    return out
```

chain — concatenate iterables lazily, one after another: a generator expression, so the tail is never touched until asked for (`chain over count()` is fine). `concat` is the strict, list-building cousin.

```
>>> list(chain([1], [2, 3]))
[1, 2, 3]
>>> take(4, chain([1, 2], count(10)))
[1, 2, 10, 11]
```

Roll your own — the definition:

```
chain = lambda *iterables: (x for it in iterables for x in it) # concat, lazily
```

9 slicing & spans

islice — at most the first *n* items of ANY iterable, lazily; the safe window onto an infinite stream. Runs short without complaint.

```
>>> list(islice(count(0), 3))
[0, 1, 2]
>>> list(islice([1, 2], 5))
[1, 2]
```

Roll your own — the definition:

```
def islice(iterable, stop): # take
    it = iter(iterable)
    for _ in range(stop):
        try:
            yield next(it)
        except StopIteration:
            return
```

take — the first *n*, as a real list. *take/drop* are the positional pair; *takeWhile/dropWhile* are the conditional pair.

```
>>> take(3, count(1))
[1, 2, 3]
>>> take(2, [7, 8, 9])
[7, 8]
```

Roll your own — the definition:

```
take = lambda n, xs: list(islice(iter(xs), n)) # works on infinite streams
```

takewhile — yield elements WHILE the predicate holds, then stop at the first failure (a generator). Contrast *filter_*: *filter_* scans everything; *takewhile* stops – on [2, 4, 5, 6], *filter_* keeps the 6, *takewhile* does not.

```
>>> list(takewhile(even, [2, 4, 5, 6]))
[2, 4]
>>> filter_(even, [2, 4, 5, 6])
[2, 4, 6]
```

Roll your own — the definition:

```
def takewhile(cond, xs): # generator: yield the leading run where cond holds
    for x in xs:
        if not cond(x):
            return
    yield x
```

takeuntil — like *takewhile(not p)*, but INCLUDES the first element where *p* holds – the do-while flavour: "read up to and including the terminator".

```
>>> list(takeuntil(lambda x: x == 3, [1, 2, 3, 4]))
[1, 2, 3]
>>> list(takeuntil(lambda x: x > 0, [5, 6]))
[5]
```

Roll your own — the definition:

```
def takeuntil(cond, xs): # like takewhile(not . cond), but INCLUDES the first hit
    for x in xs:
        yield x
        if cond(x):
            return
```

dropwhile — discard while the predicate holds, then yield EVERYTHING after the first failure, no more testing. `takewhile`’s complement.

```
>>> list(dropwhile(lambda x: x < 3, [1, 2, 3, 1]))
[3, 1]
```

Roll your own — the definition:

```
def dropwhile(cond, xs): # generator: drop the leading run where cond holds
    it = iter(xs)
    for x in it:
        if not cond(x):
            yield x
            break
    yield from it
```

takeWhile — `takewhile`, delivered as a list (capital W = list version).

```
>>> takeWhile(lambda x: x < 3, [1, 2, 3, 1])
[1, 2]
```

Roll your own — the definition:

```
takeWhile = lambda cond, xs: list(takewhile(cond, xs)) # take the leading run where cond holds,
as a list
```

dropWhile — `dropwhile`, delivered as a list.

```
>>> dropWhile(lambda x: x < 3, [1, 2, 3, 1])
[3, 1]
```

Roll your own — the definition:

```
dropWhile = lambda cond, xs: list(dropwhile(cond, xs)) # drop the leading run where cond holds,
as a list
```

span — split at the first failure: (longest satisfying prefix, the rest), computed in ONE pass so it is safe on one-shot iterators (the old two-scan version silently broke on generators – this is the fix). A lexer in one call.

```
>>> span(lambda x: x < 3, [1, 2, 3, 4, 1])
([1, 2], [3, 4, 1])
>>> span(str.isdigit, "123abc")
(['1', '2', '3'], ['a', 'b', 'c'])
```

Roll your own: materialise `xs` once (that is the one-pass guarantee), locate the first failing index with `next()` over `enumerate` – defaulting to `len(xs)` when the predicate never fails – and slice twice. The previous two-scan version ran `takewhile` and `dropwhile` separately and silently broke on one-shot generators; this is the repair.

```
def span(cond, xs): # split at first failure, ONE pass; safe on one-shot iters
    xs = list(xs)
    i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
    return (xs[:i], xs[i:])
```

break_ — `span` with the predicate negated: split where the condition first HOLDS.

```
>>> break_(lambda x: x > 2, [1, 2, 3, 1])
([1, 2], [3, 1])
```

Roll your own — the definition:

```
break_ = lambda cond, xs: span(compose(not_, cond), xs) # split at first hit (opposite of span)
```

inits — every prefix, from empty to the whole thing; `len(xs)+1` of them.

```
>>> inits([1, 2])
```

```
| [[], [1], [1, 2]]
|>>> inits("ab")
| [' ', 'a', 'ab']
```

Roll your own — the definition:

```
| inits = lambda xs: [xs[:i] for i in range(len(xs) + 1)] # Data.List: every prefix, [] first
```

tails — every suffix, whole thing first, down to empty. Substring problems start here: every substring is a prefix of some suffix.

```
|>>> tails("abc")
| ['abc', 'bc', 'c', '']
|>>> tails([1, 2])
| [[1, 2], [2], []]
```

Roll your own — the definition:

```
| tails = lambda xs: [xs[i:] for i in range(len(xs) + 1)] # every suffix; substrings start here
```

chunksOf — consecutive pieces of length *n*; the last may be short. Works on lists and strings alike since it is pure slicing.

```
|>>> chunksOf(2, [1, 2, 3, 4, 5])
| [[1, 2], [3, 4], [5]]
|>>> chunksOf(3, "abcdefg")
| ['abc', 'def', 'g']
```

Roll your own — the definition:

```
| chunksOf = lambda n, xs: [xs[i:i + n] for i in range(0, len(xs), n)] # Data.List.Split; short
| last ok
```

10 sorting & searching

sortOn — sort by a key function, evaluated once per element. TUPLE KEYS are the power move: (primary, secondary) sorts two levels at once, and negating a numeric component flips just that level – (-count, word) is "count descending, ties alphabetical", which reverse=True cannot express.

```
|>>> sortOn(len, ["bbb", "a", "cc"])
| ['a', 'cc', 'bbb']
|>>> sortOn(lambda kv: (-kv[1], kv[0]), [('b', 2), ('a', 2), ('c', 3)])
| [('c', 3), ('a', 2), ('b', 2)]
```

Roll your own — the definition:

```
| sortOn = lambda f, xs: sorted(xs, key=f) # sortOn (schwartzian, f called once per element)
```

minOn — the element with the smallest key, in $O(n)$. **sortOn** + head gives the same answer for $O(n \log n)$ – pay the sort only when you need ALL of them ordered.

```
|>>> minOn(abs, [-3, 2, 5])
| 2
|>>> minOn(snd, [('a', 9), ('b', 1)])
| ('b', 1)
```

Roll your own — the definition:

```
| minOn = lambda f, xs: min(xs, key=f) # minimumBy (comparing f): sortOn + head without the sort
```

maxOn — the element with the largest key, $O(n)$; the FIRST winner on ties, so pre-sorting the input controls

the tie-break.

```
>>> max0n(len, ["aa", "b", "cc"])
'aa'
```

Roll your own — the definition:

```
max0n = lambda f, xs: max(xs, key=f) # maximumBy (comparing f) in O(n); first wins ties
```

cmp_to_key — adapts an old-style comparator (returning negative, zero or positive) into the key= protocol. Reach for it only when a rule genuinely compares two elements against each other and no per-element key exists.

```
>>> sorted(["bb", "a"], key=cmp_to_key(lambda a, b: len(a) - len(b)))
['a', 'bb']
```

Roll your own: a tiny wrapper class implementing ONE dunder — `__lt__` defers to `cmp(a, b) < 0`. Python's sort only ever asks "is a less than b?", so that single method is enough to smuggle any comparator through the key= interface.

```
def cmp_to_key(cmp): # resurrect Python 2's cmp for sortBy
    class K:
        def __init__(self, x): self.x = x
        def __lt__(self, other): return cmp(self.x, other.x) < 0
    return K
```

sortBy — comparator sort, Python 2 style, built on `cmp_to_key`.

```
>>> sortBy(flip(sub), [1, 3, 2])
[3, 2, 1]
```

Roll your own — the definition:

```
sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp)) # comparator sort, Python 2 style
```

merge — join two ALREADY-SORTED lists into one sorted list, stably, in $O(n+m)$: the two-pointer loop at the heart of merge sort.

```
>>> merge([1, 4, 6], [2, 3, 7])
[1, 2, 3, 4, 6, 7]
>>> merge([], [1, 2])
[1, 2]
```

Roll your own: two cursors, always copying the smaller head forward; the `<=` (rather than `<`) is what makes the merge STABLE, preferring the left list on ties. When either side runs dry, glue both tails on — one of them is empty, so the `+` is harmless.

```
def merge(a, b): # merge two sorted lists, stable; the heart of merge sort
    out, i, j = [], 0, 0 # (section 13's merge_lists is this same loop on ListNodes)
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```

bisect_left — binary search over a SORTED list: the first index whose element is $\geq x$; equivalently, where x would insert to the left of its equals. If $a[i] \neq x$ there, x is absent — that check is binary search.

```
>>> bisect_left([10, 20, 20, 30], 20)
1
>>> bisect_left([10, 20, 30], 25)      # insertion point for a missing value
2
```

How it works: on a sorted list the predicate "element < x" reads True for a prefix and False for the rest – TTTTFFFF. Binary search does not hunt for x; it finds that BOUNDARY, halving the interval per probe. `bisect_left` returns the first index $\geq x$, `bisect_right` the first $> x$, and from those two facts three idioms fall out.

```
>>> xs = [10, 20, 20, 30]
>>> bisect_left(xs, 20), bisect_right(xs, 20)      # the run of 20s occupies [1, 3)
(1, 3)
```

Idiom one, membership: x is present iff the slot `bisect_left` names actually holds it. Idiom two, insertion point: that same index is where x belongs to keep the list sorted. Idiom three, counting: right minus left counts occurrences – and with two different probes, the values inside any range.

```
>>> i = bisect_left(xs, 25)
>>> i, (i < len(xs) and xs[i] == 25)              # 25 would insert at 3; absent
(3, False)
```

Why it earns its place: every threshold ladder is secretly a sorted list. Grading bands, tax brackets, rate tiers – one `bisect` replaces the `if/elif` staircase:

```
>>> grade = lambda score: "FDCBA"[bisect_right([60, 70, 80, 90], score)]
>>> grade(59), grade(60), grade(95)
('F', 'D', 'A')
```

Remember it as: `bisect` finds the boundary in a sea of Trues and Falses; everything else is reading that boundary three ways.

Roll your own: the `lo/hi` halving loop over an invariant – everything left of `lo` is $< x$, everything from `hi` rightwards is $\geq x$. One comparison, `a[mid] < x`, decides which half survives; when `lo` meets `hi`, that meeting point IS the boundary. `bisect_right` is the identical loop with `<=` in place of `<` – one character moves the boundary to the far side of the equals.

```
def bisect_left(a, x): # first index where a[i] >= x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = halve(lo + hi)
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

`bisect_right` — the first index whose element is $> x$: where x would insert to the RIGHT of its equals. right minus left counts the occurrences.

```
>>> bisect_right([10, 20, 20, 30], 20)
3
>>> xs = [10, 20, 20, 30]
>>> bisect_right(xs, 20) - bisect_left(xs, 20)
2
```

Roll your own — the definition:

```
def bisect_right(a, x): # first index where a[i] > x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = halve(lo + hi)
        if a[mid] <= x:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

11 strings

words — split on any whitespace; runs collapse, edges vanish. The inverse direction is **unwords**, and the round trip normalises spacing.

```
>>> words(" such spacing ")
['such', 'spacing']
>>> unwords(words(" a b "))
'a b'
```

Roll your own — the definition:

```
words = str.split # split on whitespace
```

unwords — join words with single spaces.

```
>>> unwords(['a', 'b', 'c'])
'a b c'
```

Roll your own — the definition:

```
unwords = " ".join # join words with single spaces
```

lines — split a string into its lines (no trailing newlines kept).

```
>>> lines("one\ntwo")
['one', 'two']
```

Roll your own — the definition:

```
lines = str.splitlines # split on newlines
```

unlines — join lines, TERMINATING each with a newline (not separating – note the trailing one).

```
>>> unlines(['a', 'b'])
'a\nb\n'
```

Roll your own — the definition:

```
unlines = lambda ls: "".join(l + "\n" for l in ls) # join lines back with trailing newlines
```

12 containers

defaultdict — a dict whose missing keys create themselves via a factory: touch `d[k]` and `factory()` is installed there. This "autovivification" makes grouping and counting one-liners. Gotcha: READING a missing key also plants it – use `'in'` or `.get` when you only want to look.

```
>>> d = defaultdict(list)
>>> d['x'].append(1)
>>> d
{'x': [1]}
>>> d2 = defaultdict(int)
>>> d2['ghost']
0
>>> 'ghost' in d2
True
```

a mere read...

...planted the key

How it works: one dunder does everything. When `d[k]` misses, Python calls `d.__missing__(k)`; this subclass responds by calling the stored factory, installing the result, and returning it. The factory runs only on a miss – and it takes NO arguments, which is why the leaf recipes in `Tree/ITree` are zero-argument lambdas.

Why it earns its place: it deletes the check-then-create dance from every grouping and counting loop. The write path becomes a bare append or `+=`, and the structure assembles itself on touch – autovivification, which section 12’s trie tools then apply recursively.

The discipline it demands: a mere READ also plants the key, so split your paths – write with `d[k]`, but ask questions with `'in'` or `.get`. The prelude’s bag operations read Counters with `.get` for exactly this reason.

```
>>> d = defaultdict(int)
>>> d['hits'] += 1                # write path: indexing is the point
>>> d.get('misses', 0), 'misses' in d  # read path: no ghost key planted
(0, False)
```

Remember it as: index to build, `.get` to ask.

Roll your own: subclass `dict` and implement one dunder – `__missing__`, which Python calls when `[k]` fails. Call the factory, install the result, hand it back; iteration, `repr` and everything else is inherited. Note the factory takes no arguments, which is why leaf recipes are zero-argument lambdas.

```
class defaultdict(dict): # dict that manufactures a value via factory() on first miss
    def __init__(self, factory=None, *args, **kw):
        super().__init__(*args, **kw)
        self.factory = factory
    def __missing__(self, key):
        if self.factory is None:
            raise KeyError(key)
        self[key] = self.factory()
        return self[key]
```

Counter — count occurrences: a `defaultdict(int)` fold over the input. Missing keys read as 0 (mind the autovivification gotcha above); rank the `.items()` with `sortOn`.

```
>>> c = Counter("abracadabra")
>>> c['a'], c['z']
(5, 0)
>>> sortOn(lambda kv: -kv[1], list(Counter("aab").items()))
[('a', 2), ('b', 1)]
```

Roll your own: a `defaultdict(int)` plus one fold – `d[x] += 1` per element. The `+=` on a missing key is autovivification doing all the work.

```
def Counter(xs): # count-by-value; a defaultdict(int) fold
    d = defaultdict(int)
    for x in xs:
        d[x] += 1
    return d
```

insertWith — `Data.Map`’s `insertWith f k v m`: insert `(k, v)`, combining with `f(new, old)` on collision – Haskell’s argument order, new value first. PERSISTENT, also like Haskell: it returns a fresh dict and leaves the argument untouched.

```
>>> insertWith(add, 'x', 4, {'x': 1})
{'x': 5}
>>> d = {'y': 2}
>>> insertWith(add, 'x', 4, d)
{'y': 2, 'x': 4}
>>> d                                # the original survives
{'y': 2}
```

Roll your own: one dict display – `**d, k: ...`: the splat copies `d` (that IS the persistence), and the new key’s value is `f(v, d[k])` on collision (new value first, Haskell’s order) or plain `v`. Compare `fromListWith`, which applies the same rule but mutates, because there the dict is its own private accumulator.

```
insertWith = lambda f, k, v, d: {**d, k: f(v, d[k]) if k in d else v} # Data.Map insertWith:
    PERSISTENT
    # (fresh dict); f(new, old)
```

fromListWith — THE dict-building fold: pour (key, value) pairs into a dict, combining collisions with $f(\text{new}, \text{old})$ – Haskell’s order, like `insertWith`. Keys keep first-seen order.

```
>>> fromListWith(add, [(c, 1) for c in "aab"])
{'a': 2, 'b': 1}
>>> fromListWith(flip(add), [('a', [1]), ('b', [2]), ('a', [3])])
{'a': [1, 3], 'b': [2]}
```

The order gotcha, faithfully Haskell’s: because f receives (new, old), grouping with a bare `concat` REVERSES each group – the classic `Data.Map` surprise. Combine as `flip(add)` (old + new) to keep arrival order, or use plain `add` where order cannot matter (counts).

```
>>> fromListWith(add, [(len(w), [w]) for w in ["hi", "ox", "sun"]])
{2: ['ox', 'hi'], 3: ['sun']}
>>> fromListWith(flip(add), [(len(w), [w]) for w in ["hi", "ox", "sun"]])
{2: ['hi', 'ox'], 3: ['sun']}
```

Why it earns its place – one function, three monoids: choosing f is choosing what the dict MEANS. Addition over $(k, 1)$ pairs counts; old + new over $(k, [v])$ pairs groups; `max` merges bags. That is the whole family: `insertWith` is one collision, `fromListWith` a list of them, `unionWith` two dicts’ worth (where f receives (a’s value, b’s value)).

The recognition cue: whenever a loop builds a dict with an if-else on key existence, you are hand-rolling this fold. Name the pairs, name the monoid, and the loop disappears.

Remember it as: grouping, counting, and merging are one fold – whose combiner hears the NEW value first.

Roll your own: an empty dict and one loop applying the collision rule per pair – $d[k] = f(v, d[k])$ if the key exists, else v . It is `insertWith` inlined and IN-PLACE: mutation is fine here because the dict being built is the fold’s own accumulator, invisible until returned.

```
def fromListWith(f, pairs): # Data.Map fromListWith: THE dict-building fold; f(new, old)
    d = {} # like insertWith -- so grouping with concat REVERSES each
    for k, v in pairs: # group (the classic Haskell gotcha); use add for counts
        d[k] = f(v, d[k]) if k in d else v
    return d # Counter == fromListWith(add) over (x, 1);
    # grouping == fromListWith(++) over (k, [v])
```

unionWith — merge two dicts, combining SHARED keys with f ; a’s keys keep their order, b’s new keys follow. Choosing f is choosing a monoid: `max` gives bag union, `+` gives a summing merge.

```
>>> unionWith(max, {'a': 1, 'b': 5}, {'b': 2, 'c': 3})
{'a': 1, 'b': 5, 'c': 3}
>>> unionWith(add, {'a': 1, 'b': 2}, {'b': 3, 'c': 4})
{'a': 1, 'b': 5, 'c': 4}
```

Roll your own: two splats – `**a, **...`: a copied whole, then overlaid with b’s items after each has been through the collision rule, $f(a[k], v)$ when k is shared, plain v when it is new. The asymmetry is deliberate: a’s keys keep their positions, and only keys new to b append at the end.

```
unionWith = lambda f, a, b: {**a, **{k: f(a[k], v) if k in a else v for k, v in b.items()}}
# Data.Map unionWith: f(a's value, b's value) on shared keys;
# bag_union == unionWith(max), merge-sum == unionWith(add)
```

group — runs of CONSECUTIVE equal elements, as bare lists (`[a]`), no keys – `Data.List`’s `group`.

```
>>> group("aaabbc")
[['a', 'a', 'a'], ['b', 'b'], ['c']]
>>> group("abab")
```

```
| [['a'], ['b'], ['a'], ['b']]
```

Roll your own — the definition:

```
| group = partial(groupBy, lambda a, b: a == b) # Data.List group: runs of equals, [[a]] -- no
      keys
```

groupBy — runs by an equivalence predicate; each newcomer is compared with eq against the run's FIRST element (span-based, exactly Haskell's groupBy), not against its neighbour – a subtle and deliberate fidelity.

```
| >>> groupBy(lambda a, b: a == b, [1, 1, 2])
| [[1, 1], [2]]
| >>> groupBy(lambda a, b: b - a <= 1, [1, 2, 3, 10, 11])
| [[1, 2], [3], [10, 11]]
```

The word that matters is CONSECUTIVE: a new run starts whenever eq against the run's first element fails, so 'abab' yields four runs. Not a weakness – a different question: runs, streaks and compression care about position, and these are the only tools asking about it.

```
| >>> [(run[0], len(run)) for run in group("aaabbbbcc")] # run lengths
| [('a', 3), ('b', 4), ('c', 2)]
```

For CATEGORIES – position irrelevant – either sort first, so equals become adjacent, or use fromListWith, which never cared about position at all. Choosing between them is stating what your key means.

```
| >>> [(run[0], len(run)) for run in group(sorted("abab"))]
| [('a', 2), ('b', 2)]
```

Remember it as: group answers "how long are the runs?"; fromListWith answers "what are the piles?".

Roll your own: one pass in which each element either joins the current run or starts a fresh one. The test is eq(out[-1][0], x) – against the run's FIRST element, not its last – which is exactly how Haskell's span-based groupBy behaves, and the detail that makes non-transitive predicates (like "within 1 of") agree with it.

```
| def groupBy(eq, xs): # Data.List groupBy: runs of CONSECUTIVE elements; each new
|   out = [] # element is compared via eq against the run's FIRST element,
|   for x in xs: # exactly like Haskell (span-based) -- not its neighbor
|       if out and eq(out[-1][0], x):
|           out[-1].append(x)
|       else:
|           out.append([x])
|   return out
```

bag_union — multisets (bags) are Counters; union takes the LARGER count per key. Key order follows set union, so compare with == rather than printing.

```
| >>> bag_union(Counter("aab"), Counter("abb")) == {'a': 2, 'b': 2}
| True
```

Roll your own — the definition:

```
| bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
```

bag_inter — intersection: the SMALLER count per shared key, zero counts dropped. Note the implementation reads with .get, never [] – indexing a Counter would autovivify zeros into it.

```
| >>> bag_inter(Counter("aab"), Counter("ab")) == {'a': 1, 'b': 1}
| True
| >>> bag_inter(Counter("aa"), Counter("bb")) == {}
| True
```

Roll your own — the definition:

```
| bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a.get(k, 0), b.get(k,
```

```
| 0))}) # min count per shared key
```

bag_diff — difference, clipped at zero: what remains of a after removing b.

```
| >>> bag_diff(Counter("aab"), Counter("ab")) == {'a': 1}
| True
```

Roll your own — the definition:

```
| bag_diff = lambda a, b: {k: a[k] - b[k] for k in a if a[k] > b.get(k, 0)} # clipped at 0
```

bag_sub — inclusion: is bag a contained in bag b, multiplicities included? The ransom-note test.

```
| >>> bag_sub(Counter("ab"), Counter("aabb"))
| True
| >>> bag_sub(Counter("aab"), Counter("ab"))
| False
```

Roll your own — the definition:

```
| bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items()) # a <= b (inclusion)
```

Tree — nested defaultdicts to a FIXED depth, with a leaf factory at the bottom. `Tree(1, list)` is the grouping dict; `Tree(2, int)` is a sparse 2-D counting table. Indexing autovivifies the whole path.

```
| >>> t = Tree(1, list)
| >>> t['evens'].append(2)
| >>> t
| {'evens': [2]}
| >>> m = Tree(2, int)
| >>> m['a']['b'] += 1
| >>> m
| {'a': {'b': 1}}
```

Roll your own — the definition:

```
| Tree = lambda depth, leaf: (defaultdict(leaf) if depth == 1 # fixed-depth nested defaultdict;
|                               else defaultdict(lambda: Tree(depth - 1, leaf))) # leaves default
| to leaf()
```

ITree — the INFINITE tree: every branch is another ITree, so any depth autovivifies on touch. This is the trie: index letter by letter and the branch builds itself.

```
| >>> t = ITree()
| >>> t['x']['y']['z'] = 9
| >>> t
| {'x': {'y': {'z': 9}}}
```

How it works: the definition is a knot – `ITree = lambda: defaultdict(ITree)`. Every missing key manufactures another ITree, which will do the same, without end. Indexing therefore GROWS the tree: touching `t['c']['a']['t']` conjures the whole branch. A trie in one line, built from nothing but ___missing___.

```
| >>> t = ITree()
| >>> for w in ["cat", "car"]: setpath(t, list(w) + ['$'], w)
| >>> sorted([leaf for p, leaf in paths(t) if p[-1] == '$'])
| ['car', 'cat']
```

Why it earns its place: hierarchies – tries, directory trees, nested groupings – stop needing setup code. Writing is indexing (or `setpath`, which also plants a leaf); harvesting is `paths`. The one edged rule: reads autovivify too, so QUESTIONS go through `getpath`, never bare indexing – `getpath`'s entry demonstrates the ghost branch this avoids.

Versus `Tree(depth, leaf)`: `Tree` is the finite cousin – fixed depth, with a real leaf factory (`list`, `int`) at the

bottom instead of more tree. `Tree(1, list)` is the grouping dict; `ITree` is for depth you cannot know in advance, like words.

Remember it as: a dict that dreams up more of itself on demand.

Roll your own: `ITree = lambda: defaultdict(ITree)` mentions itself – legal, because the lambda body only runs on a MISS, by which time the name is bound. `Tree` bottoms the same recursion out at a fixed depth by handing the last level a real leaf factory instead of more tree.

```
| ITree = lambda: defaultdict(ITree) # infinitely-nestable defaultdict: a tree that grows on  
|     access
```

paths — fold a whole `Tree/ITree` into a flat list of (keypath, leaf) pairs, in insertion order. The read-everything companion to `setpath`'s write-one-path.

```
| >>> t = ITree()  
| >>> setpath(t, ['a', 'b'], 1); setpath(t, ['a', 'c'], 2)  
| >>> paths(t)  
| [[(['a', 'b'], 1), (['a', 'c'], 2)]]
```

How it works: structural recursion with two arms, exactly the case-arms-as-fold story. A non-dict (or empty node) is a LEAF: report the single row (`[]`, leaf). A dict is a BRANCH: recurse into every subtree and prepend the branching key to each keypath that comes back. The whole tree flattens into (keypath, leaf) rows, insertion-ordered, losslessly.

Why it earns its place: building a trie is the easy half; the value is HARVESTING it. `autocomplete` filters rows under a prefix, `deepest_directory` takes `maxOn` of keypath length, `leaves` reads the second column. One cata, many harvests – you never write tree-walking code again.

```
| >>> t = ITree()  
| >>> setpath(t, ['a', 'b'], 1); setpath(t, ['c'], 2)  
| >>> paths(t)  
| [[(['a', 'b'], 1), (['c'], 2)]]  
| >>> maxOn(lambda row: len(row[0]), paths(t))  
| (['a', 'b'], 1)
```

Remember it as: `paths` turns a tree back into rows; `setpath` turns rows into a tree.

Roll your own: recursion on one question – is this still a dict? A leaf reports a single row with an empty path; a branch recurses into every child and conses its key onto every path that comes back. Depth equals tree height, comfortably inside Python's limit for sane tries.

```
| def paths(t): # cata for Tree/ITree: [(keypath, leaf)]  
|     # recursion depth = tree height (word length / len(nums)) -- well under the ~1000 limit  
|     if not isinstance(t, dict) or not t: # leaf = non-dict value, or empty node  
|         return [[[], t]]  
|     return [[([k] + p, leaf) for k, sub in t.items() for p, leaf in paths(sub)]]
```

leaves — just the leaf values of `paths`.

```
| >>> t = ITree()  
| >>> setpath(t, ['a'], 1); setpath(t, ['b'], 2)  
| >>> leaves(t)  
| [1, 2]
```

Roll your own — the definition:

```
| leaves = compose(partial(map_, snd), paths) # every leaf value in a Tree/ITree, in path order
```

setpath — write a leaf value at the end of a key path, autovivifying the branch on the way. Caution: writing at an interior key CLOBBERS the subtree below it.

```
| >>> t = ITree()  
| >>> setpath(t, ['a', 'b'], 7)
```

```
>>> t
{'a': {'b': 7}}
>>> setpath(t, ['a'], 0)           # clobbers {'b': 7}
>>> t
{'a': 0}
```

Roll your own: walk all but the last key by PLAIN INDEXING – on an ITree that autovivifies the branch as you go – then assign at the final key, the single write. `getpath` is the mirror image with the safety catch on: `isinstance` and `'in'` checks instead of indexing, so reading plants nothing.

```
def setpath(t, ks, v): # write leaf v at key path ks (autovivifies interior)
    for k in ks[:-1]:
        t = t[k]
    t[ks[-1]] = v # NB: clobbers any subtree already at ks
```

getpath — read along a key path WITHOUT autovivifying, returning a default when the path is absent. This is why it exists: reading a trie with `[]` would plant ghost branches.

```
>>> t = ITree()
>>> setpath(t, ['a', 'b'], 7)
>>> getpath(t, ['a', 'b']), getpath(t, ['a', 'z'], default=0)
(7, 0)
>>> 'z' in t['a']           # no ghost branch was planted
False
```

Roll your own — the definition:

```
def getpath(t, ks, default=None): # read along ks WITHOUT autovivifying
    for k in ks:
        if not isinstance(t, dict) or k not in t:
            return default
        t = t[k]
    return t
```

deque — a double-ended queue built from two stacks, all four end operations amortised $O(1)$ (elements migrate between stacks only when one runs dry). This is the BFS frontier; a plain list's `pop(0)` is $O(n)$.

```
>>> d = deque([1, 2, 3])
>>> d.appendleft(0); d.append(4)
>>> d.popleft(), d.pop()
(0, 4)
>>> len(d), list(d)
(3, [1, 2, 3])
```

How it works: two plain lists, nose to nose – `_out` holds the front (reversed), `_in` holds the back. `append` pushes onto `_in`, `appendleft` onto `_out`, both $O(1)$. The trick is popping an EMPTY side: the other list is reversed across in one $O(n)$ migration, and popping is $O(1)$ again.

Why that still counts as $O(1)$: amortization. Each element migrates at most once between being pushed and popped, so n operations cost $O(n)$ in total – constant per operation on average, which is what the workload's big-O cares about. A bare Python list cannot say this: `pop(0)` shifts every element, every single time.

```
>>> q = deque()
>>> for x in [1, 2, 3]: q.append(x)
>>> q.popleft(), q.popleft()      # first pop pays the migration; the next is free
(1, 2)
```

Why it earns its place: BFS. The frontier must yield its OLDEST member (`popleft`) while newcomers join at the back (`append`) – `levelorder` and `grid_path` are this class doing its one job.

Remember it as: two stacks facing each other make a queue; the reversal bill is paid once per element.

Roll your own: two plain lists nose to nose, each end appending and popping its own list at $O(1)$. All the cleverness is in the pop guard: when your side is empty, reverse the OTHER list across in one go. Each element makes that crossing at most once between entering and leaving, so the occasional $O(n)$ reversal amortises to $O(1)$ per operation.

```
class deque: # two-stack; all four ends amortized O(1)
    def __init__(self, xs=()):
        self._in, self._out = [], list(reversed(list(xs)))
    def append(self, x): self._in.append(x)
    def appendleft(self, x): self._out.append(x)
    def pop(self):
        if not self._in:
            self._in, self._out = list(reversed(self._out)), []
        return self._in.pop()
    def popleft(self):
        if not self._out:
            self._out, self._in = list(reversed(self._in)), []
        return self._out.pop()
    def __len__(self): return len(self._in) + len(self._out)
    def __iter__(self): return iter(self._out[::-1] + self._in)
```

dsu — disjoint-set union (union-find) over n elements, with path halving. `find` gives a set's representative; union merges and returns `False` when the two were ALREADY connected – which is exactly a cycle detector (Kruskal's test).

```
>>> find, union = dsu(4)
>>> union(0, 1), union(1, 2)
(True, True)
>>> union(0, 2) # already connected: a cycle
False
>>> find(0) == find(2), find(0) == find(3)
(True, False)
```

How it works: `parent[]` links every element toward its set's ROOT, and `find` follows the chain up. The subtle line is `parent[x] = parent[parent[x]]` – path HALVING: every `find` re-points each visited node at its grandparent, so chains shrink as a side effect of being walked. That one line makes operations effectively constant time (inverse Ackermann, for the curious).

Why it earns its place: dynamic connectivity. "Are these two in the same group yet?" under a stream of merges is awkward for every other structure and trivial here. And union returning `False` – already connected – IS cycle detection: Kruskal's spanning tree is built by skipping every `False`.

```
>>> find, union = dsu(5)
>>> [union(a, b) for a, b in [(0, 1), (1, 2), (3, 4), (0, 2)]]
[True, True, True, False]
>>> len({find(i) for i in range(5)}) # two components remain
2
```

Remember it as: `find` walks to the root, halving as it goes; a `False` union is a cycle caught.

Roll your own: `parent[]` as the forest and two closures instead of a class. `find`'s loop carries the entire trick in one line – `parent[x] = parent[parent[x]]` re-points every visited node at its grandparent (path halving), flattening chains as a side effect of walking them. `union` is `find` twice plus one root re-point, answering `False` when the roots already agree.

```
def dsu(n): # union-find, path halving; union -> False if already joined
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
        x = parent[x]
```

```

        return x
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        parent[ra] = rb
        return True
    return find, union

```

heappush — push onto a min-heap kept in a plain list (sift-up). The smallest element is always `h[0]`. For a MAX-heap, push negated values.

```

>>> h = []
>>> for x in [5, 1, 8]: heappush(h, x)
>>> h[0]
1

```

How it works: the flat list IS a binary tree – `h[i]`'s children live at $2i+1$ and $2i+2$ – kept under one invariant: every parent $\leq$ its children. `heappush` appends at the first free leaf and sifts UP, swapping with its parent while smaller; `heappop` moves the last leaf to the root and sifts DOWN toward the smaller child. Each walks one root-to-leaf path: $O(\log n)$.

Why the invariant is enough: nothing is fully sorted – siblings sit in any order – yet `h[0]` is ALWAYS the minimum, because the root beats its children, who beat theirs. You pay $\log n$ per operation for a permanently readable minimum; draining the heap is `heapsort`; pushing negatives turns min into max.

```

>>> h = []
>>> for x in [7, 2, 9, 4]: heappush(h, x)
>>> h[0]
2
>>> [heappop(h) for _ in range(4)]
[2, 4, 7, 9]

```

Why it earns its place: "repeatedly take the best next thing" – Dijkstra's frontier, `running_median`'s two halves, any streaming top-k. When next-best is needed many times while the data keeps changing, the heap beats re-sorting every time.

Remember it as: a flat list, a parent-beats-children promise, and $\log n$ repairs.

Roll your own: append at the first free leaf, then sift UP – while the parent at $(i - 1) // 2$ is larger, swap and climb. The index arithmetic IS the tree; watch the array after four pushes:

```

>>> h = []
>>> for x in [7, 2, 9, 4]: heappush(h, x)
>>> h
[2, 4, 9, 7]
# root 2; children 4 and 9; 7 pushed down under 4

```

```

def heappush(h, x): # min-heap on a plain list: sift-up
    h.append(x)
    i = len(h) - 1
    while i and h[halve(i - 1)] > h[i]:
        h[halve(i - 1)], h[i] = h[i], h[halve(i - 1)]
        i = halve(i - 1)

```

heappop — remove and return the minimum (sift-down). Popping repeatedly drains in sorted order – that is `heapsort`.

```

>>> h = []
>>> for x in [5, 1, 8]: heappush(h, x)
>>> [heappop(h) for _ in range(3)]
[1, 5, 8]
>>> g = []

```

```
>>> for x in [3, 9, 5]: heappush(g, -x)
>>> -heappop(g)          # max-heap via negation
9
```

Roll your own: swap the root with the last leaf, pop it off the end, then sift DOWN – repeatedly swap with the SMALLER of the two children (choosing the smaller is the classic bug site) until neither child is smaller. One root-to-leaf path each way: $O(\log n)$.

```
def heappop(h): # min-heap on a plain list: sift-down
    h[0], h[-1] = h[-1], h[0]
    top = h.pop()
    i, n = 0, len(h)
    while True:
        s, l, r = i, 2*i + 1, 2*i + 2
        if l < n and h[l] < h[s]: s = l
        if r < n and h[r] < h[s]: s = r
        if s == i:
            return top
        h[i], h[s] = h[s], h[i]
        i = s
```

13 nodes

TreeNode — the LeetCode binary tree: val, left, right, with None for absent children.

```
>>> t = TreeNode(2, TreeNode(1), TreeNode(3))
>>> t.left.val, t.val, t.right.val
(1, 2, 3)
```

Roll your own — the definition:

```
class TreeNode: # the LeetCode binary tree
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
```

tfold — the tree catamorphism: recursively combines each node’s value with the already-folded left and right results, z standing in for empty subtrees. EVERY structural tree computation is one choice of f and z – the five traversal/measure functions below are all tfold one-liners.

```
>>> t = TreeNode(2, TreeNode(1), TreeNode(3))
>>> tfold(lambda v, l, r: v + l + r, 0, t)    # sum
6
>>> tfold(lambda v, l, r: max(v, l, r), 0, t) # max
3
```

How it works: the tree catamorphism – structural recursion with a contract. Two arms, exactly like a case expression: the empty tree answers z; a node answers f(value, left_result, right_result), both results computed by the same recursion. Hand it (f, z) and any question about a tree becomes one line – the five traversals and both measures in section 13 are all tfold with different arms.

Why it earns its place: it separates WALKING from DECIDING. The recursion is written once, correct forever; you only ever author the arms. Even tree TRANSFORMATION fits – mirror a tree by swapping the child results while rebuilding:

```
>>> t = TreeNode(2, TreeNode(1), TreeNode(3))
>>> mirror = tfold(lambda v, l, r: TreeNode(v, r, l), None, t)
>>> inorder(mirror)
[3, 2, 1]
```

The one caveat: recursion depth equals tree height – fine for balanced and interview-sized trees; say a sentence out loud about adversarially skewed ones.

Remember it as: *z* is the empty-tree arm, *f* is the node arm – the fold walks, you decide.

Roll your own: one line of structural recursion – *z* when the tree is *None*, else *f* over the node’s value and the two recursive results.

Roll your own: one line of structural recursion – *z* when the tree is *None*, else *f* over the node’s value and the two recursive results. Every traversal and measure in this section is that line with a different *f* plugged in.

```
| tfold = lambda f, z, t: z if t is None else f(t.val, tfold(f, z, t.left), tfold(f, z, t.right))
```

inorder — left, node, right; on a binary SEARCH tree this is sorted order.

```
| >>> inorder(TreeNode(2, TreeNode(1), TreeNode(3)))
| [1, 2, 3]
```

Roll your own — the definition:

```
| inorder = partial(tfold, lambda v, l, r: l + [v] + r, []) # left, root, right
```

preorder — node first, then children: the copy/serialise order.

```
| >>> preorder(TreeNode(2, TreeNode(1), TreeNode(3)))
| [2, 1, 3]
```

Roll your own — the definition:

```
| preorder = partial(tfold, lambda v, l, r: [v] + l + r, []) # root, left, right
```

postorder — children first, then node: the delete/evaluate order.

```
| >>> postorder(TreeNode(2, TreeNode(1), TreeNode(3)))
| [1, 3, 2]
```

Roll your own — the definition:

```
| postorder = partial(tfold, lambda v, l, r: l + r + [v], []) # left, right, root
```

tdepth — the height of the tree.

```
| >>> tdepth(TreeNode(2, TreeNode(1, TreeNode(0)), TreeNode(3)))
| 3
```

Roll your own — the definition:

```
| tdepth = partial(tfold, lambda _, l, r: 1 + max(l, r), 0) # height of the tree
```

tsize — the number of nodes.

```
| >>> tsize(TreeNode(2, TreeNode(1), TreeNode(3)))
| 3
```

Roll your own — the definition:

```
| tsize = partial(tfold, lambda _, l, r: 1 + l + r, 0) # number of nodes in the tree
```

levelorder — breadth-first values, level by level: the deque earning its keep. Compare preorder on the same tree to see DFS vs BFS.

```
| >>> t = TreeNode(1, TreeNode(2, TreeNode(4)), TreeNode(3))
| >>> levelorder(t)
| [1, 2, 3, 4]
| >>> preorder(t)
| [1, 2, 4, 3]
```

Roll your own: seed a deque with the root, then loop – popleft, record the value, append the children that exist. FIFO order IS breadth-first; swap the deque for a list used as a stack and the identical loop turns into depth-first.

```
def levelorder(t): # BFS: the deque earning its keep
    out, q = [], deque([t] if t else [])
    while len(q):
        n = q.popleft()
        out.append(n.val)
        if n.left: q.append(n.left)
        if n.right: q.append(n.right)
    return out
```

ListNode — the LeetCode singly-linked list: val and next.

```
>>> n = ListNode(1, ListNode(2))
>>> n.val, n.next.val
(1, 2)
```

Roll your own — the definition:

```
class ListNode: # the LeetCode linked list
    def __init__(self, val=0, nxt=None):
        self.val, self.next = val, nxt
```

from_list — build a linked list from a Python list: literally foldr(ListNode, xs, None). foldr’s f takes (element, accumulator), and ListNode(val, nxt) has exactly that shape, so the last element is wrapped first and each earlier one points at the list built so far.

```
>>> to_list(from_list([1, 2, 3]))
[1, 2, 3]
```

Roll your own — the definition:

```
from_list = lambda xs: foldr(ListNode, xs, None) # build; foldr of ListNode
```

to_list — walk a linked list back into a Python list; the round-trip partner of from_list, and the way to make node results printable. An unfoldr: the seed is the node, each step yields (node.val, node.next), and None ends it.

```
>>> to_list(from_list("ab"))
['a', 'b']
>>> to_list(None)
[]
```

Roll your own — the definition:

```
to_list = compose(list, partial(unfoldr, lambda n: None if n is None else (n.val, n.next))) #
    ListNode chain -> list
```

reverse_list — reverse in place with the three-pointer walk; prev is a fold accumulator wearing pointers.

```
>>> to_list(reverse_list(from_list([1, 2, 3])))
[3, 2, 1]
```

Roll your own: the three-pointer walk compressed into one simultaneous assignment – node.next, prev, node = prev, node, node.next. The right-hand side is evaluated in full before any assignment lands, which is what saves the pointer you are about to overwrite.

```
def reverse_list(node): # three-pointer; prev is a fold accumulator
    prev = None
    while node:
        node.next, prev, node = prev, node, node.next
```

```
    return prev
```

middle — the middle node by fast/slow pointers (fast moves twice per step); on even lengths, the SECOND middle.

```
>>> middle(from_list([1, 2, 3])).val
2
>>> middle(from_list([1, 2, 3, 4])).val
3
```

Roll your own: fast advances two hops for slow's one, so when fast falls off the end, slow stands at the middle. The loop condition (fast and fast.next) is precisely what selects the SECOND middle on even lengths.

```
def middle(node): # fast/slow: second middle for even lengths
    slow = fast = node
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    return slow
```

has_cycle — Floyd's tortoise and hare: the fast pointer laps the slow one if and only if a cycle exists; O(1) space.

```
>>> has_cycle(from_list([1, 2, 3]))
False
>>> n = ListNode(1); n.next = n          # a self-loop
>>> has_cycle(n)
True
```

Roll your own: the same fast/slow walk, but watching for the pointers to MEET — on a cycle the fast runner laps the slow one; on a straight list it simply falls off. O(1) space, no visited set.

```
def has_cycle(node): # Floyd: fast laps slow iff a cycle exists
    slow = fast = node
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False
```

merge_lists — merge two SORTED linked lists, using the dummy-head idiom to avoid special-casing the first node.

```
>>> to_list(merge_lists(from_list([1, 3]), from_list([2, 4])))
[1, 2, 3, 4]
```

Roll your own: the dummy-head idiom — a throwaway node to hang the result from, so appending the first real node needs no special case; walk both lists copying the smaller head, glue the survivor, return dummy.next.

```
def merge_lists(a, b): # merge two sorted lists; dummy-head idiom
    dummy = tail_ = ListNode()
    while a and b:
        if a.val <= b.val:
            tail_.next, a = a, a.next
        else:
            tail_.next, b = b, b.next
        tail_ = tail_.next
    tail_.next = a or b
    return dummy.next
```

14 grids & windows

neighbors4 — the in-bounds orthogonal neighbours of cell (r, c) in an R-by-C grid. Bounds checking lives HERE, so BFS bodies stay clean.

```
>>> sorted(neighbors4(0, 0, 2, 2))      # corner: two neighbours
[(0, 1), (1, 0)]
>>> sorted(neighbors4(1, 1, 3, 3))      # centre: four
[(0, 1), (1, 0), (1, 2), (2, 1)]
```

Roll your own: yield each in-bounds delta of (r, c) – the bounds check lives HERE so every caller's BFS body stays clean. **neighbors8** is the same generator over a 3x3 delta grid, with (dr or dc) excluding the centre cell.

```
def neighbors4(r, c, R, C): # in-bounds orthogonal neighbors
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        if 0 <= r + dr < R and 0 <= c + dc < C:
            yield r + dr, c + dc
```

neighbors8 — the same plus diagonals.

```
>>> len(list(neighbors8(1, 1, 3, 3)))
8
>>> sorted(neighbors8(0, 0, 2, 2))
[(0, 1), (1, 0), (1, 1)]
```

Roll your own — the definition:

```
def neighbors8(r, c, R, C): # ... plus diagonals
    for dr in (-1, 0, 1):
        for dc in (-1, 0, 1):
            if (dr or dc) and 0 <= r + dr < R and 0 <= c + dc < C:
                yield r + dr, c + dc
```

longest_window — the sliding-window skeleton: it grows the right edge one element at a time (calling push), shrinks the left edge while your valid() says the window is illegal (calling shed), and tracks the best length. YOU supply the state as closures; the skeleton does the two-pointer bookkeeping. Longest-substring-without-repeats and longest-run-under-a-budget are both three closures away.

```
>>> seen = []
>>> longest_window("abcabb", lambda: len(seen) == len(set(seen)),
...               seen.append, lambda c: seen.pop(0))
3
>>> w = []
>>> longest_window([2, 1, 3, 4], lambda: sum(w) <= 6,
...               w.append, lambda x: w.pop(0))
3
```

How it works: inversion of control. The skeleton owns the two pointers – it grows hi one element per step (calling push), then shrinks lo while your valid() reports the window broken (calling shed per evicted element) – and it tracks the best length ever seen legal. You own only the STATE, held in closures over variables sitting next to the call.

Designing the closures is answering one question: "what makes a window ILLEGAL, and what must I track to know?" Distinct characters -> a duplicate count. A budget -> a running sum. At most k of something -> a counter. push and shed are that state's increment and decrement; valid is the legality test.

```
>>> w = []
>>> longest_window("aabcbb", lambda: len(w) == len(set(w)), w.append, lambda c: w.pop(0))
3
```

Why closures and not a class: the state lives exactly as long as the call, sits in scope where you can read

it, and needs no ceremony. Functions carrying state – the same muscle as curry’s captured argument, here working a two-pointer job.

Remember it as: the skeleton slides the window; your three closures say what legal means.

Roll your own: `hi` sweeps forward exactly once, `add()` greeting each newcomer; an inner while shrinks `lo`, `shed()` by `shed()`, until `valid()` approves again. Every element is added once and removed at most once – the entire two-pointer $O(n)$ argument, visible in two loops.

```
def longest_window(xs, valid, push, shed): # sliding-window skeleton; state lives in the
    closures
    lo = best = 0
    for hi in range(len(xs)):
        push(xs[hi])
        while not valid(): # shrink until legal again
            shed(xs[lo])
            lo += 1
        best = max(best, hi - lo + 1)
    return best
```

15 control

memo — an unbounded memoizer as a plain decorator (no parentheses): results are cached by the argument tuple, so each distinct call computes once. Keyword arguments fold into the key; the cache is exposed as `.cache`.

```
>>> @memo
... def fib(n): return n if n < 2 else fib(n - 1) + fib(n - 2)
>>> fib(30)
832040
>>> fib.cache[(10,)]
55
>>> len(fib.cache)                                # one entry per distinct argument
31
```

How it works: the decorator wraps `f` with a dict keyed on the arguments. First call computes and stores; every repeat is a lookup. That is the entire mechanism – and why arguments must be `HASHABLE`: lists become tuples before they can be keys.

Why it earns its place: memoized recursion IS dynamic programming. Write the honest recurrence – the answer for state (i, j) in terms of smaller states – decorate it, and the cache is your DP table, filled in dependency order with no index bookkeeping. The exposed `.cache` lets you SEE the table: its size is the state count, exactly the number complexity analysis wants said out loud.

```
>>> @memo
... def ways(r, c):                                # lattice paths to (r, c), moving right/down
...     return 1 if r == 0 or c == 0 else ways(r - 1, c) + ways(r, c - 1)
>>> ways(3, 3)
20
>>> len(ways.cache)                                # the 4x4 grid minus (0,0), never needed
15
```

Versus `functools.lru_cache`: the standard library’s version adds eviction (`maxsize`); `memo` is deliberately unbounded, because DP wants every state kept – and it is nine lines you can retype in any interview that bans imports.

Remember it as: recursion states the truth; the cache makes it affordable; `.cache` is the table.

Roll your own: a dict captured in a closure, keyed by the argument tuple (keyword arguments folded in as a `frozenset` when present – tuples and `frozensets` because keys must be hashable). Compute on miss, look up

on hit, and hang the dict on the wrapper as .cache so the DP table stays inspectable.

```
def memo(f): # unbounded memoizer; enough for DP (no eviction by design)
    cache = {}
    def wrapped(*args, **kw):
        key = (args, frozenset(kw.items())) if kw else args
        if key not in cache:
            cache[key] = f(*args, **kw)
        return cache[key]
    wrapped.cache = cache # peek at the DP table if curious
    return wrapped
```

until — iterate f from x until the predicate holds, returning the first satisfying value: a fixed-point loop with no recursion-depth limit. The state can be a tuple carrying whatever the loop must remember.

```
>>> until(lambda x: x > 100, lambda x: x * 2, 1)
128
>>> collatz = lambda s: (s[0] // 2 if even(s[0]) else 3 * s[0] + 1, s[1] + 1)
>>> until(lambda s: s[0] == 1, collatz, (6, 0)) # 8 steps to reach 1
(1, 8)
```

How it works: keep applying f until p approves – iteration to a goal or a FIXED POINT, as a loop rather than recursion, so depth is unbounded. The state can be anything; a tuple lets one loop carry several facts at once, like the collatz example's (value, steps).

Why it earns its place: some processes have no list to fold over – they run "until stable". Newton's isqrt converges; bellman_ford relaxes every edge until the distance map stops changing (the textbook fixed point; g7's hint calls it by name). until states such loops declaratively: here is the step, here is done.

```
>>> until(lambda n: n * n >= 30, succ, 0) # smallest n with n squared >= 30
6
>>> until(lambda xs: xs == sorted(xs), sorted, [3, 1, 2])
[1, 2, 3]
```

Remember it as: while-not-done, promoted to an expression that returns its answer.

Roll your own: while not p(x): x = f(x); return x. Iteration promoted to a value-returning expression – and a loop rather than recursion on purpose, because a fixed point may take unboundedly many steps.

```
def until(cond, f, x): # until cond f x -- loop, not recursion (unbounded depth)
    while not cond(x):
        x = f(x)
    return x
```